

Neuroevolution of Compact Search Heuristics in Sokoban

An Honors Thesis Presented by Yeonghun Lee
To the Department of Computer Science
Advised by Professor Gary B. Parker

Connecticut College
New London, Connecticut
May 2026

Abstract

This thesis investigates whether neuroevolution can produce heuristic functions for A* search on Sokoban that are drastically more parameter-efficient than a conventional gradient-trained baseline. Sokoban is a combinatorial planning puzzle that is both NP-hard and PSPACE-complete, and solving it at scale requires an accurate cost-to-go heuristic to guide search. The standard approach fixes a neural network architecture and trains it by gradient descent; this thesis investigates whether evolutionary search over network topology can find heuristics that are competitive in accuracy while using orders of magnitude fewer parameters.

Four heuristic approaches are implemented and compared on the medium-difficulty split of the Boxoban benchmark, a dataset of procedurally generated 10×10 Sokoban levels released by Google. A parameter-free Manhattan-distance heuristic, summing per-box minimum distance to a goal, anchors the minimum baseline. A CNN cost-to-go network adapted from DeepCubeA, with roughly 2 million parameters trained by AdamW on cost-to-go labels derived from Breadth-First Search solution trajectories, anchors the upper-limit baseline. NEAT (NeuroEvolution of Augmenting Topologies) evolves both the topology and the weights of a feed-forward heuristic from scratch using the same labels, growing complexity only where it improves prediction accuracy. CoDeepNEAT coevolves populations of network modules and blueprint topologies, combining evolutionary architecture search with per-genome gradient-descent weight training.

The headline finding is that neuroevolved heuristics fall on opposite sides of the Manhattan minimum baseline: pure NEAT, despite searching over both topology and weights, fails to clear it, while CoDeepNEAT clears it using roughly $7.7\times$ fewer parameters than the CNN upper-limit baseline. CoDeepNEAT is therefore the only neuroevolved method in this thesis that beats the parameter-free analytical baseline; the addition of Adam gradient descent on each assembled genome is what closes the gap that pure NEAT cannot. The results show that neuroevolution *can* beat a parameter-free analytical baseline on Boxoban, but only when evolutionary architecture search is paired with gradient-based weight optimization on inputs of this dimensionality.

Contents

Introduction	2
Background and Motivation	2
Contributions	4
1 Background	6
1.1 A* Search	6
1.2 Convolutional Neural Networks	7
1.3 NeuroEvolution of Augmenting Topologies	8
1.3.1 Encoding and Historical Markings	9
1.3.2 Structural Mutations	9
1.3.3 NEAT in Game-Playing Agents	10
1.4 CoDeepNEAT: Evolving Deep Architectures using NEAT	10
2 History of Sokoban	12
2.1 Gameplay	12
2.2 History	13
2.3 Reception	13
3 Sokoban in Academia	15
3.1 NP-hard	15
3.2 PSPACE-complete	19
3.3 Automatic Generation of Sokoban Puzzles	20
4 Solving Sokoban	24
4.1 Early Attempts at Solving Sokoban	24
4.2 Reinforcement Learning Approaches	25
4.2.1 I2A and Boxoban	25
4.2.2 Forward-Backward Reinforcement Learning	26
4.2.3 Learning Generalized Reactive Policies	27
4.3 DeepCubeA	28
5 Methodology	30
5.1 Boxoban	30
5.2 Heuristic Approaches	30
5.2.1 Manhattan Distance (Minimum Baseline)	31
5.2.2 CNN Baseline (Upper-Limit Baseline)	31
5.2.3 NEAT	32
5.2.4 CoDeepNEAT	33
5.3 Forward BFS Trajectories and Cost-to-Go Labels	34
5.4 Training Metrics	36
6 Results	38
6.1 Experiments and Results	38
6.1.1 Manhattan Distance (Minimum Baseline)	38
6.1.2 CNN Baseline (Upper-Limit Baseline)	39
6.1.3 NEAT Discussion	41
6.1.4 CoDeepNEAT Discussion	44

6.1.5	Interpretation of Findings	45
7	Conclusion	46
7.1	Future Work	47
A	Supplementary Results	49
A.1	Manhattan Distance Baseline Results	49
A.2	CNN Baseline Results	50
A.3	CoDeepNEAT Results	52
A.4	NEAT Hyperparameter Sweep	52

Introduction

Background and Motivation

Sokoban is a combinatorial planning puzzle invented by Hiroyuki Imabayashi in Japan in 1981. A player occupies a grid of cells containing walls, boxes, and goal positions, and must push every box onto a goal by moving up, down, left, or right. The rules are simple, but its puzzles can be extremely complex. Sokoban has been shown to be both NP-hard [1] and PSPACE-complete [2], placing it at the upper limits of complexity for discrete planning problems.

The difficulty arises from two main properties. First, the game is irreversible. Once a box is pushed against a wall or into a corner that is not a goal position, it can never be retrieved since the player cannot pull the box. This is a deadlock that forces the player to fully reset (or undo, depending on the version of the game). The existence of deadlocks constrains the agent from exploring the space stochastically, as a single incorrect move can invalidate all previous moves. This poses a significant challenge for exploration-based agents, which are the current standard of reinforcement learning. Second, rewards are sparse. For a box to be pushed to the correct position, the agent must first perform a long list of actions that may or may not have a clear reward. An agent can always push the box toward a goal, but this does not guarantee that the box can be pushed onto the goal from that specific direction. The lack of accurate feedback means there is no gradient signal for simple training-based agents. Together, these properties make Sokoban a canonical stress-test for planning agents.

A Brief History of Sokoban Solvers

Early approaches to automated Sokoban solving, developed through the 1990s and early 2000s, were largely rule-based. Solvers such as YASS [3] and Sokoban++ [4] relied on handcrafted heuristics, deadlock tables, and macro-moves. These a priori heuristics were designed by human experts to capture common patterns of solutions in Sokoban puzzles

by avoiding deadlocks. These systems achieved impressive results on small curated puzzle sets such as XSokoban [5], but they required significant domain knowledge to implement.

The 2010s brought reinforcement learning methods to Sokoban. Deep Q-Networks and policy gradient agents [6] were applied to the puzzle, with some imagination-augmented architectures [7] using a learned world model to simulate future states before committing to an action. These model-based agents demonstrated that a neural network could implicitly learn to plan ahead without handcrafted rules, but they remained sample-inefficient and struggled on the most complex instances.

A more powerful paradigm emerged from work on the Rubik’s Cube. Agostinelli et al. [8] showed that by training a deep neural network on a reverse trajectory that expanded the state space from the solved state, the network could learn a generalized cost-to-go function that could serve as the heuristic function in an A* search, enabling an agent to solve the Rubik’s Cube optimally without any domain-specific knowledge. The reverse trajectory is generated by scrambling the Rubik’s Cube from the solved state with random moves and labeled by the number of moves applied. Subsequent work extended this framework to Sokoban and demonstrated that the same cost-to-go approach, guided by A* search, can solve highly complex puzzle instances including Sokoban-like puzzles [9].

The Boxoban Benchmark

Despite these advances, no study has applied the cost-to-go A* framework to Boxoban, a large-scale Sokoban dataset released by Google consisting of nine million procedurally generated 10×10 levels. Boxoban was created to benchmark agents that must plan over long horizons on a diverse distribution of puzzles, specifically to stress-test imagination-based planning models: approaches that use a learned world model to internally simulate sequences of future states before selecting an action [7]. In such models, the agent unrolls the world model forward from the current state, generating imaginary trajectories that it encodes alongside real observations to inform its policy. Although Boxoban was originally framed as a stress test for imagination-based agents, it is a challenging testbed

for reinforcement learning agents more broadly: the long planning horizons, irreversible moves, and sparse reward structure of Sokoban make it a valid benchmark for any method that must learn to plan, and the fixed 10×10 grid size yields a compact $5 \times 10 \times 10$ multi-hot state representation that any architecture can consume directly without resizing or per-level preprocessing. The scale of nine million unique levels prevents any solver from memorizing solutions and instead demands genuine generalization across the full distribution of puzzle configurations.

Contributions

This thesis makes four primary contributions, organized as a pair of reference baselines bracketing two neuroevolved heuristics.

First, we establish a parameter-free Manhattan-distance heuristic, summing each box’s minimum Manhattan distance to a goal cell, as the *minimum* baseline. Because the heuristic is provably admissible under unit-cost pushes, it is the classical analytical reference any learned heuristic must beat to justify the cost of training.

Second, we implement a CNN baseline heuristic with four layers of convolution and 2 million trainable parameters, adapted from the DeepCubeA cost-to-go formulation, as the *upper-limit* baseline. The loss function was modified to punish overestimation of value more than underestimation to encourage training an admissible heuristic.

Third, we apply NEAT [10] to evolve a feed-forward heuristic network from a minimal starting topology. NEAT searches simultaneously over network topology and weights, adding nodes and connections only where doing so improves the fitness signal, and uses speciation to protect structural innovations from premature elimination.

Fourth, we apply CoDeepNEAT [11] to coevolve populations of dense-dropout modules and blueprint topologies, with each assembled genome trained by gradient descent for a fixed number of epochs per generation. This hybrid approach combines the architectural flexibility of evolutionary search with the weight-optimization efficiency of gradient descent, producing heuristics that are more lightweight than the CNN baseline while main-

taining competitive accuracy.

All learned heuristics are trained on cost-to-go labels derived from Breadth-First Search solution trajectories over the Boxoban dataset and evaluated on unseen levels by mean absolute error and A* solve rate, with the Manhattan and CNN baselines providing the lower and upper reference points for the neuroevolved methods.

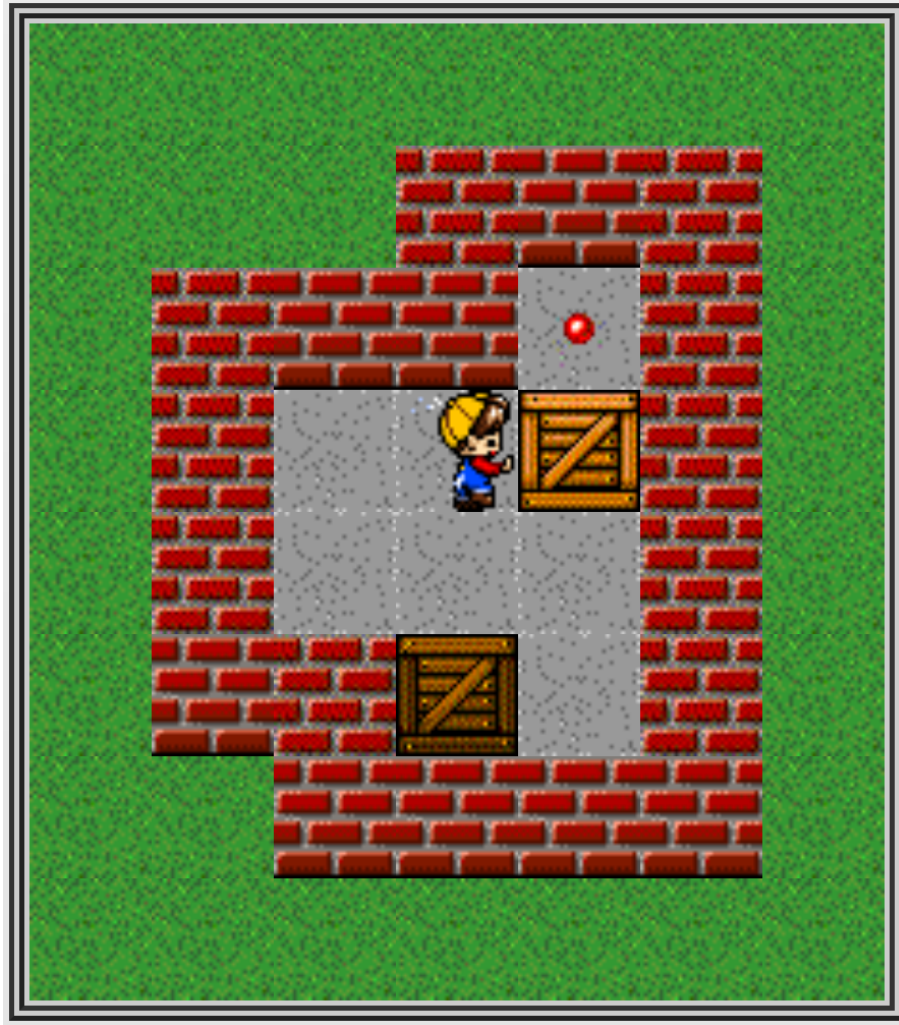


Figure 1: A Sokoban puzzle. The player (yellow hardhat) must push the box (brown square) onto the goal position (red dot) without trapping it against a wall or corner. Screenshot from <https://www.sokoban.jp/>.

Background

Before delving into the specific methodologies and results of this thesis, it is important to establish the foundational concepts and prior work that inform the approach taken in this thesis. This chapter provides an overview of the key algorithms and techniques that underpin the Sokoban-solving agents, including A* search, convolutional neural networks (CNNs), NeuroEvolution of Augmenting Topologies (NEAT), and CoDeepNEAT. Each section contextualizes these methods within the broader landscape of AI research and highlights their relevance to the Sokoban-solving framework.

A* Search

A* is a best-first graph search algorithm that finds the least-cost path from an initial state to a goal state [12]. Unlike Breadth-First Search, which expands nodes uniformly by depth, A* prioritizes nodes using an evaluation function that combines the cost already incurred with an estimate of the remaining cost to the goal:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the exact cost of the path from the start to node n , and $h(n)$ is a heuristic estimate of the remaining cost from n to the goal. A* maintains a priority queue sorted by $f(n)$, always expanding the lowest-cost node first.

The correctness of A* depends on the admissibility of the heuristic. A heuristic h is *admissible* if it never overestimates the true cost to the goal: $h(n) \leq h^*(n)$ for all n , where $h^*(n)$ is the true minimum cost. Under this condition, A* is guaranteed to find an optimal solution with sufficient computational budget.

The practical performance of A* is determined largely by the quality of h . A tight heuristic directs the search along the optimal path with minimal node expansions, while a

weak one causes the frontier to grow combinatorially. In domains with large state spaces, such as Sokoban, the choice of heuristic is critical for tractability.

Convolutional Neural Networks

Convolutional Neural Networks (CNNs) [13] are a class of deep learning models designed to automatically and adaptively learn spatial hierarchies of features from input data. While early architectures focused on character recognition, the breakthrough of AlexNet [14] demonstrated that deep CNNs could scale to complex visual tasks through the use of GPUs and non-linear activations.

While standard multilayer perceptrons (MLPs) struggle with the high dimensionality of raw grid data, CNNs leverage local receptive fields and weight sharing to maintain spatial relationships, making them particularly effective for game boards like Sokoban. The fundamental operation of a CNN is the discrete convolution. In the context of a convolutional layer, the input is typically represented as a 3D multi-hot tensor $I \in \mathbb{R}^{H \times W \times C}$, where H and W are the spatial dimensions and C represents the depth or feature planes (e.g., specific object types in a Sokoban state). The layer consists of a set of learnable kernels K , where each kernel has a smaller spatial extent than the input but spans the same depth.

The output of a 3D convolutional layer, known as a feature map $\mathcal{O}^{(k)}$, is computed by sliding a cubic kernel over the input volume. For a 3D input I and a 3D kernel K , the value at position (i, j, l) of the output feature map is given by the cross-correlation operation:

$$(I * K)_{i,j,l} = \sum_{m=0}^{h-1} \sum_{n=0}^{w-1} \sum_{p=0}^{c-1} I_{i+m,j+n,l+p} \cdot K_{m,n,p}$$

In a deep learning context, this is typically augmented with a learnable bias $b^{(k)}$ and passed through a non-linear activation function σ , such as the Rectified Linear Unit (ReLU):

$$a_{i,j,l}^{(k)} = \sigma \left(\sum_{m=0}^{h-1} \sum_{n=0}^{w-1} \sum_{p=0}^{c-1} I_{i+m,j+n,l+p} \cdot \mathcal{W}_{m,n,p}^{(k)} + b^{(k)} \right)$$

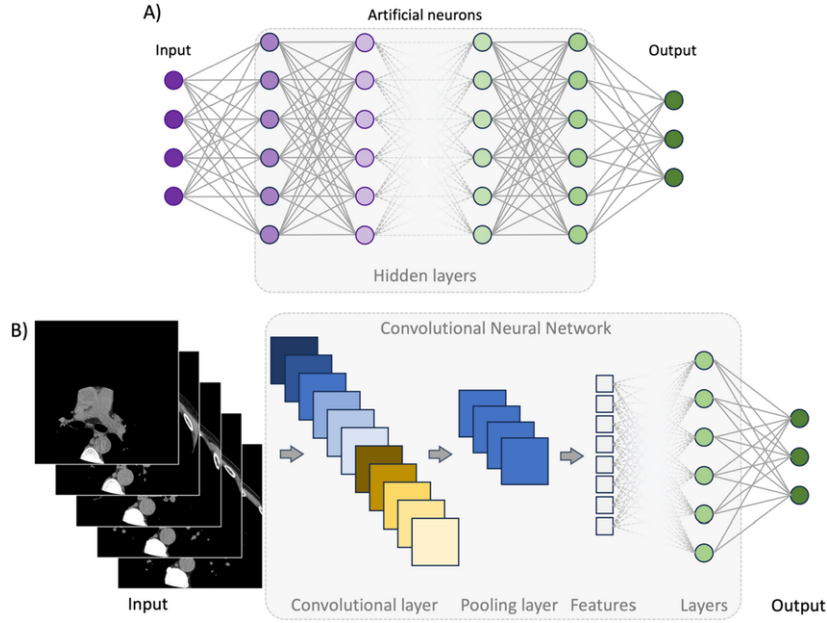


Figure 1.1: Architecture of a typical Deep Learning CNN, showing the transition from input through convolutional and pooling layers to the final output [15].

This process allows the network to achieve translation invariance; because the same weights ($\mathcal{W}^{(k)}$) are applied across the entire input, the network can recognize a "box" or a "target" regardless of its specific coordinates on the Sokoban grid.

The result is a trainable network that performs a linear transformation from the input to the feature space, followed by a non-linear transformation to the output space for various tasks ranging from classification to regression.

NeuroEvolution of Augmenting Topologies

NeuroEvolution of Augmenting Topologies (NEAT), introduced by Stanley and Miikkulainen [10], is an evolutionary algorithm that optimizes both the weights and the topology of artificial neural networks simultaneously. Traditional neuroevolution fixes the network architecture in advance and uses evolutionary or gradient-based methods only to adjust the weights. NEAT relaxes this constraint, allowing evolution to add neurons and connections incrementally, starting from a minimal network and growing complexity through mutation.

Encoding and Historical Markings

NEAT represents each network as a genome consisting of two gene lists: node genes and connection genes. A connection gene encodes a directed edge between two nodes, carrying a weight, an enable flag, and an *innovation number*, a global counter incremented each time a new structural mutation occurs anywhere in the population. Innovation numbers serve as historical markings that identify which structural mutations arose at the same point in evolutionary history, making it possible to align genomes of different topologies for crossover without requiring computationally expensive topological matching.

During crossover, innovation numbers let NEAT align two genomes in linear time and classify each gene as *matching* (present in both parents), *disjoint*, or *excess*, which lets two structurally different networks recombine without disrupting functional substructures. The same matching/disjoint/excess counts later feed the compatibility distance used for speciation, so a single bookkeeping device solves both problems at once.

Structural Mutations

NEAT introduces two structural mutations in addition to the standard weight perturbation:

- *Add connection*: a new connection is inserted between two previously unconnected nodes, with a randomly initialized weight and a new innovation number.
- *Add node*: an existing connection is split by inserting a new node. The original connection is disabled, and two new connections are added; one from the source node to the new node with weight 1, and one from the new node to the target with the original weight. This initialization preserves the network’s existing behavior while providing a scaffold for further adaptation.

Because new structural mutations initially degrade fitness before they can be refined by evolution, NEAT protects innovation through *speciation*: genomes are grouped into species based on a compatibility distance measure computed from the number of disjoint

and excess genes and the average weight difference of matching genes. Fitness sharing within species ensures that novel structures are evaluated relative to topologically similar individuals rather than competing immediately against well-tuned incumbents.

NEAT in Game-Playing Agents

NEAT has demonstrated strong performance in sequential decision-making and game-playing tasks. Togelius and Lucas applied NEAT to evolve controllers for simulated car racing, demonstrating its capacity to discover reactive policies across continuous state spaces [16, 17]. More broadly, neuroevolution has been shown to match or exceed gradient-based methods on many game domains, particularly those with deceptive gradient landscapes or sparse rewards, where evolutionary exploration of topology provides an advantage [18].

CoDeepNEAT: Evolving Deep Architectures using NEAT

The broader claim that neuroevolution can match human-designed architectures at competitive scale is supported by CoDeepNEAT (Coevolution DeepNEAT), introduced by Mikkilainen et al. [11]. CoDeepNEAT extends NEAT to the problem of designing full deep learning architectures by introducing a two-level coevolutionary structure: a population of *modules* (small subgraphs, each representing a repeatable network component such as a convolutional block) and a population of *blueprints* (higher-level graphs whose nodes point to module species). During fitness evaluation, a blueprint is instantiated by replacing each of its nodes with a module drawn from the corresponding species, assembling a complete deep network that is then trained by gradient descent and scored by validation performance.

This bilevel coevolution mirrors the modular structure of successful hand-designed networks such as GoogLeNet and ResNet, which are built from repeated blocks, but discovers that structure automatically rather than by human intuition. Because small mutations to a module propagate to every blueprint node that references it, CoDeepNEAT can explore

structurally diverse and deep architectures more efficiently than evolving a monolithic network would allow.

On the CIFAR-10 image classification benchmark, CoDeepNEAT achieved a test error of 7.3%, comparable to the 6.4% reported by the Bayesian hyperparameter optimization baseline of Snoek et al. Notably, the best evolved network trained significantly faster than the human-designed counterpart: it reached 20% test error in 12 epochs versus more than 30 for the baseline, and converged around epoch 120 versus over 200. For language modeling on the Penn Treebank dataset, 25 generations of neuroevolution over LSTM connectivity improved test perplexity by 5% over a vanilla LSTM by discovering a feedback skip connection between memory cells of two LSTM layers. This discovery of an innovative network architecture was possible only because the search was not constrained to a fixed topology, allowing evolution to explore structural mutations that gradient descent over a fixed architecture could not have found.

These results establish that NEAT-based evolutionary search over architecture space can discover both competitive performance and structurally novel solutions without expert intervention. For this thesis the key implication is one of scope: CoDeepNEAT operates at the scale of millions of parameters and full deep networks, yet the same underlying evolutionary mechanism applies equally to the small heuristic networks required for Sokoban A* search.

The most striking demonstration of the compactness potential of neuroevolution is Cuccu, Togelius, and Cudré-Mauroux’s *Playing Atari with Six Neurons* [19], which showed that a neuroevolved network of only 6–18 neurons, paired with a co-evolved compact state representation, matches or exceeds the performance of deep reinforcement learning agents using millions of parameters on several Atari benchmarks. Taken together, CoDeepNEAT and the six-neuron result mark the range of what neuroevolution can achieve: at the large end, it matches expert-designed deep networks; at the small end, it discovers solutions orders of magnitude more compact than gradient-trained baselines.

History of Sokoban

Sokoban is a widely successful puzzle game where the goal is to push boxes around in a warehouse to position the boxes in correct places. However, the positions of the boxes determine the mobility of other boxes and the player. In some cases, the player may accidentally push a box into a corner, rendering it completely immobile [20]. Due to these restrictions, players are required to plan ahead of moving to successfully solve each level.

The game was invented in 1981 by Hiroyuki Imabayashi as a hobby and was commercially released in 1982. Since its release, the series has sold over 4.1 million copies worldwide, with over 40 official games and countless clones, enabled by the simplistic gameplay design.

Gameplay

Sokoban is a top-down puzzle game in which the player controls a 2D character in a warehouse with several boxes that the player can push. Some tiles are marked as storage locations where the boxes in the level need to be placed. In order to complete the level, all boxes need to be placed on designated tiles [21]. The levels are generally surrounded by tight walls, and the boxes are as large as the player. This often means that the boxes can block paths, and the player often needs to relocate a box in order to reach certain areas in the level. This is made more difficult by the fact that the player can only push the boxes and cannot pull them. If a box is pushed against a wall, the player can only move the box tangent to the wall, as they cannot push away from the wall [22]. If the box is pushed into a corner, the box cannot be moved for the same reason [23]. If this is not part of the solution, the player reaches a dead end, and the only way to progress is to reset the current level or undo the previous move.

History

The first iteration of the game ran on the NEC-8001 as a hobby project by Hiroyuki Imabayashi [24]. The game was primarily inspired by *Aldebaran #1*, created by Hudson Soft, in which the player pushed objects to block radiation [25, 26]. However, Imabayashi envisioned a more difficult game in which objects could become stuck at specific places, a game mechanic less explored in *Aldebaran #1* due to its spacious levels [27]. The hobby project became a commercial project when a salesman came across it in a store owned by Imabayashi’s parents-in-law. Since its original launch on the NEC PC-8801 [28], the game has been re-released many times across various platforms, from handheld gaming consoles like the Game Boy [29] to consumer computers like the Apple II [30] and Windows [31], and most recently the Nintendo Switch and PlayStation 4 [32].

Reception

Sokoban instantly became the favorite of puzzle enthusiasts [33], with over 25,000 copies sold by July 1984 following the initial release in Japan [34]. It was quickly followed by multiple successful releases in Japan, which prompted a global release. The U.S. release *Soko-Ban* sold over 50,000 copies by mid-September 1988 [30]. By 2018, the series had sold over 4.1 million copies worldwide [35].

Sokoban was praised for its compelling, simple gameplay combined with difficult puzzles requiring a high level of planning, comparable in depth to Shogi [23]. The popularity and difficulty combined soon led to a study on understanding the nature of Sokoban as a problem from the computational complexity theory point of view.

The difficulty was soon studied with computational complexity theory by Fryers et al., and the result was published in 1995 in *Eureka* [1]. Sokoban was found to be NP-hard, which indicates that any NP problem can be encoded into a Sokoban puzzle in polynomial time. In other words, theoretically, finding a generic solution to all Sokoban puzzles indicates that the logic required to solve the puzzle can be applied to any other

easier NP problem, proving that any NP problem is also a P problem, which is tractable to solve. While it is widely believed that NP is not P, searching for a solution to an NP-hard problem has the academic significance that its solution may be applied to some other specific problems. Similarly to how AlexNet triggered the Deep Learning revolution in the 2010s while attempting to solve an image classification problem [14], academic inquiry continually seeks compelling problems whose solutions yield insights that extend beyond their original scope [36]. For this reason, Sokoban is an academically interesting problem. The next chapter discusses the variety of methodologies that researchers have been applying to solve Sokoban.

Sokoban in Academia

NP-hard

The preceding chapter briefly introduced the definition of NP-hard problems and the academic significance of Sokoban as a proxy for generic NP-hard problems. This section presents the proof of Fryers et al. that Sokoban is NP-hard.

Before presenting the proof, we first define a P problem as a problem that can be solved in at most polynomial time. NP problems are ones for which the existence of a solution can be verified in polynomial time, though the solution itself cannot necessarily be derived that quickly. Any P problem is in NP because if we can derive a solution in polynomial time, we can also verify it in polynomial time. It is unknown whether all NP problems are in P . NP -hard problems are ones to which any other NP problem can be reduced. An NP -hard problem that is also in NP itself is an NP -complete problem. The simplest NP -complete problem is the Boolean Satisfiability problem: for a given boolean expression $B(x_1, x_2, \dots, x_n)$ that includes $\neg$, $\wedge$, and $\vee$, is there an assignment of $x_1, x_2, \dots, x_n$ that makes B true?

If Sokoban can answer this question, then the game is as hard as Boolean Satisfiability, which is an NP -hard problem, and is therefore an NP -hard problem itself. To establish this, the researchers first demonstrated that simple boolean expressions can be encoded as playable Sokoban levels; “a level has a solution if and only if the expression can be true, and such that we can construct (in polynomial time) a valuation from a solution and vice versa” [1]. Successfully encoding a boolean expression that includes all of $\neg$, $\wedge$, and $\vee$ would show that Sokoban can express any propositional formula in this fragment.

Figure 3.1 shows a level with one entrance on the left and one exit on the right. The player can travel from the entrance to the exit by pushing the barrel (marked with a black circle) to the right and, after going through, back to the left in order to clear the path



Figure 3.1

to the exit. Not only does this clear the path for the player, but it also places the barrel back on the goal position, marked with a square. However, the player cannot travel from the exit to the entrance because the player can only push the box to the left, which would block the path and render the box immobile. This structure is called a *turnstile*.



Figure 3.2

Figure 3.2 shows a *1-gate* that can only be traveled once by the player, and only in one direction, from left to right. This is because the left barrel needs to be pushed to the left to its goal position to be considered solved, and this action must precede the movement of the right barrel, which, after its push to the left, would prevent the left barrel from being moved. This gate can be placed in parallel to create an *n-gate*, which allows traversal from left to right n times.

Using a combination of *n-gates* and *turnstiles*, we will construct the following boolean expression:

$$B(x, y, z) = (z \wedge \{(\neg y \wedge \neg z) \vee (\neg x \vee y)\})$$

The same boolean expression can be expressed as a graph of nodes (Figure 3.3) where the player can only travel from left to right.

For a given set of x, y, z , each node is traversable only if the expression inside is true.

If the player can start from the left-end of the graph and reach the right-end of the

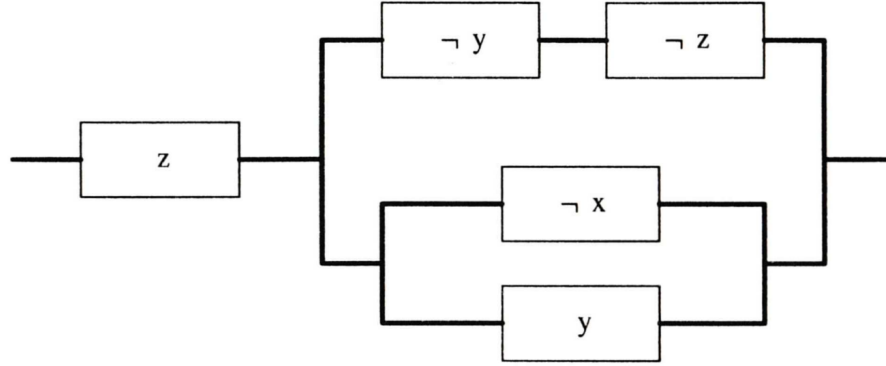


Figure 3.3

graph, $B(x, y, z)$ is true.

For example, if $x = 0, y = 0, z = 1$, it would go through the nodes z and $\neg x$ to reach the right-end of the graph.

This graph can then be recreated with the previous Sokoban level sections by defining each node as follows:

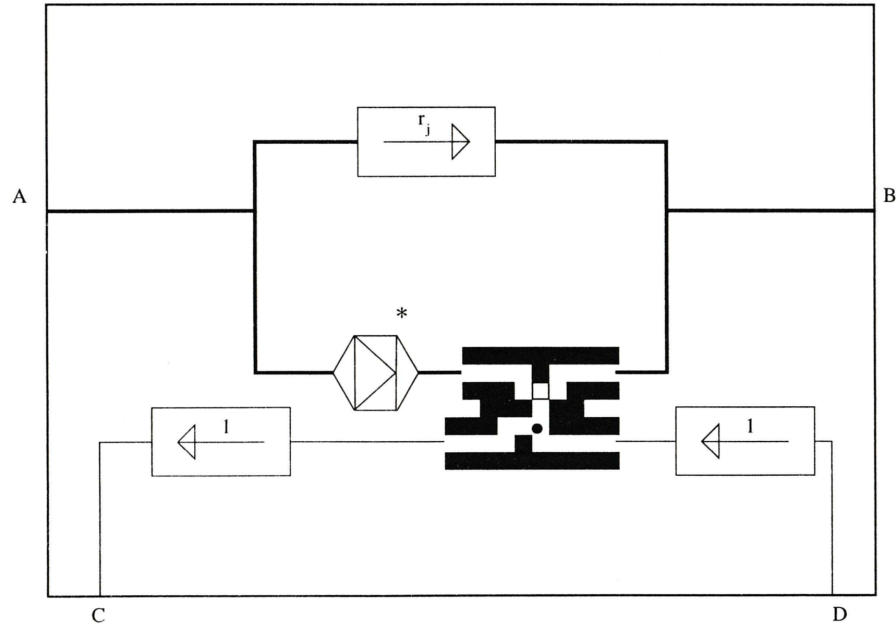


Figure 3.4

The node has entrances A, D and exits B, C . The player can traverse the node on the bold line through either the r_j -gate (an n -gate that can be traveled through r_j times)

or the *turnstile*. Notice that if the player enters D and leaves through C , it blocks the *turnstile* path above and forces the player to go through the r_j -gate path, as the barrel will be pushed upwards into the *turnstile* path and become stuck there. The bold path and the faint path are separate because, even though the barrel can be pushed away, it would cause the barrel to get stuck in a non-goal position. This node will be incorporated into the node graph from Figure 3.3 in a way that blocks the bold path of the x node when $x = 0$, blocks $\neg y$ when $y = 1$, and so on.

To show that B has a solution, the player has to be able to complete the node graph implemented in Sokoban shown in Figure 3.5. (Figure 3.5 also uses a crossover implemented in Sokoban, which is omitted in this thesis for brevity.)

Suppose $B(x, y, z)$ as implemented in Figure 3.5 is true (solvable with all boxes on their goal positions) and there exists a set of inputs $(x_1, x_2, \dots, x_n)$ (which is x, y, z in our expression) that makes B true. With this condition, the player can solve B by following the steps below:

1. From ‘start’, travel the path x if x is true and $\neg x$ otherwise, and so on.
2. The player reaches P for the first time. From here, travel the bold paths using the *turnstile* path of each node without using the r_j -gates, which can be done because B is true. The player reaches Q.
3. Traverse the faint paths not used in the first cycle to return to P.
4. From P, use the bold paths to reach Q, using the r_j -gates of the set of nodes that connect P to Q.
5. The player has now fully solved the path between ‘start’ and P. The remaining combinations of the bold paths that connect P to Q can be traveled by using the bottom bold line that has no node.
6. After traveling through the $(r + 1)$ -gate $r + 1$ times, the player can place the barrel in the R section to complete the entire level.

This demonstrates that Sokoban can successfully encode an NP -hard problem and is therefore an NP -hard problem itself. Readers can choose to complete Figure 3.5 by hand with the truth table below:

x	y	z	$\neg x$	$\neg y$	$\neg z$	$\neg y \wedge \neg z$	$\neg x \vee y$	Inner Term	$B(x, y, z)$
0	0	0	1	1	1	1	1	1	0
0	0	1	1	1	0	0	1	1	1
0	1	0	1	0	1	0	1	1	0
1	0	0	0	1	1	1	0	1	0
0	1	1	1	0	0	0	1	1	1
1	0	1	0	1	0	0	0	0	0
1	1	0	0	0	1	0	1	1	0
1	1	1	0	0	0	0	1	1	1

Table 3.1: Truth table for the Boolean function $B(x, y, z)$

PSPACE-complete

The preceding section established that Sokoban is an NP -hard problem, following the proof of Fryers et al. [1, 37]. In addition to being an NP -hard game, Sokoban was also found to be PSPACE-complete [2]. A PSPACE problem is a decision problem that can be solved by a Turing machine using a polynomial amount of memory space. If every problem in PSPACE can be reduced to a given problem X , then X is PSPACE-hard. A problem is PSPACE-complete if and only if it is in PSPACE and is PSPACE-hard.

Culberson demonstrated that Sokoban is a PSPACE-complete problem by recreating a finite-tape Turing machine in Sokoban [2]. To create the emulator, the researcher relied on two basic properties of the game. The first is the notion of unrecoverable configuration. This is when it is not possible to move a box to its goal position, either because the box was placed in a dead-end location or because the goal position is impossible to reach due to its placement. The second is the fact that the player's movement is also bound by the geometry of the puzzle. Several gadgets are then introduced, similarly to the NP -hard

proof. Each gadget is a section of a Sokoban puzzle. These gadgets are combined into Cellular Units, each representing one cell of the finite Turing machine’s tape. Culberson proves that the puzzle created from these Cellular Units has a solution if and only if the emulated Turing machine halts in an “accept” state.

Automatic Generation of Sokoban Puzzles

One of the earliest subsequent strands of academic research on Sokoban focused on generating Sokoban puzzles. The researchers framed the work as an attempt to generate art with computers, arguing that the creation of art by computer is an important target of artificial intelligence [21]. Through the automatic generation of Sokoban puzzles, Murase et al. laid the foundation for generating datasets for Sokoban agents [21].

The researchers achieved automatic generation of Sokoban levels through a three-step methodology. First, candidate problems are generated randomly from a prototype and three templates. Next, unsolvable candidates are removed by a Sokoban solver. Finally, trivial or uninteresting candidates are removed by the evaluator. Templates are 2×3 , 2×2 , and 3×3 empty spaces, used to encourage the generation of multiple paths inside the space. After tiles are placed (Figure 3.6), there are some tiles where a box can never be placed. These tiles are marked as GOAL_AVOID (Figure 3.7). Goal-candidate tiles that are in corners are marked as COR_GOAL, and the next goal is placed on goal-candidate tiles that are not dead ends and not COR_GOAL. The dead-end ADD_AVOID areas are recalculated after every placement.

After the generation of the level, a Breadth-First Search is used to solve the puzzle. While the BFS solver is unable to find solutions to puzzles with long sequences, it can still effectively search for solutions with short sequences [1]. Puzzles that cannot be solved are pruned in this step.

A key contribution of that work is the metric used to determine whether a generated puzzle is non-trivial and interesting. The evaluation program aims to evaluate artistic value using the following criteria:

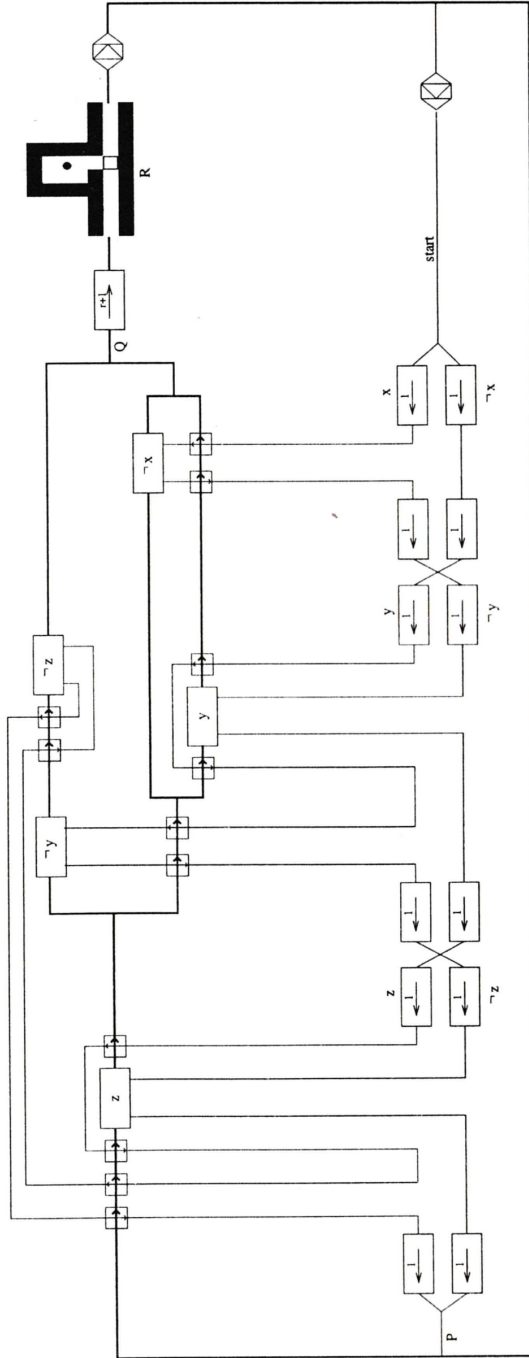


Figure 3.5

- Length of solution sequence: if a problem has a very short solution sequence, it is trivial and is rejected. When the length of a problem's solution sequence is less than seven, the problem is rejected.
- Number of changes in pushing direction during the solution sequence: in interesting problems, the keeper has to change the direction of pushing objects very often. If the number of changes is

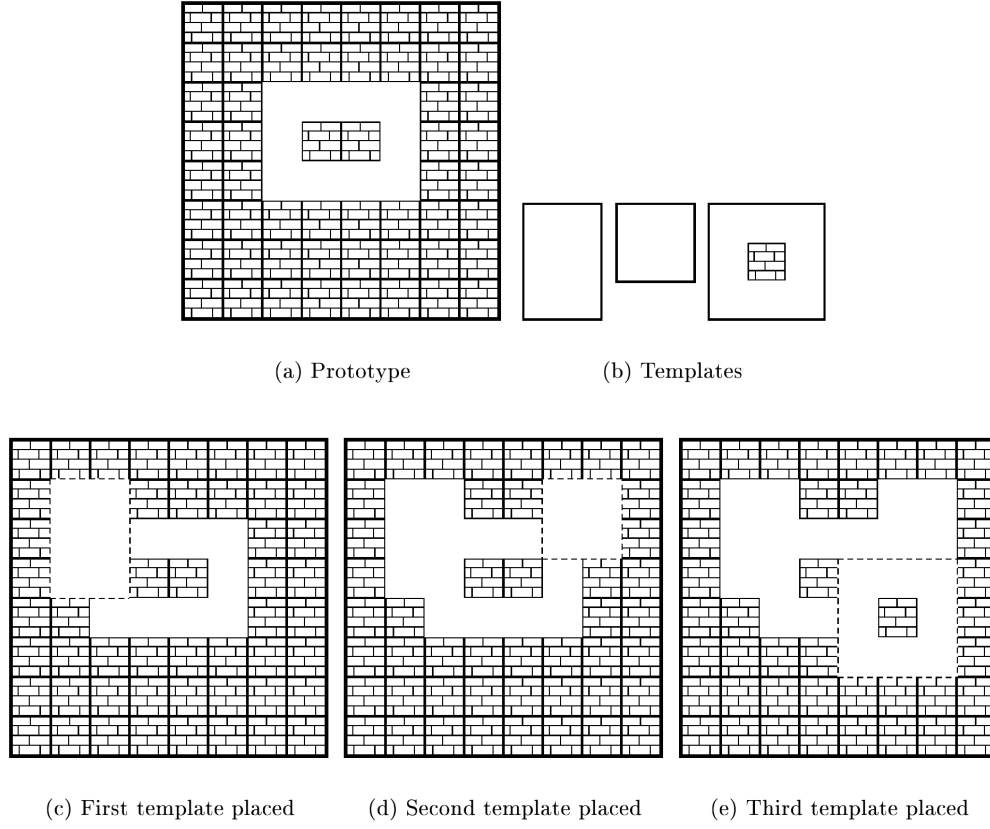
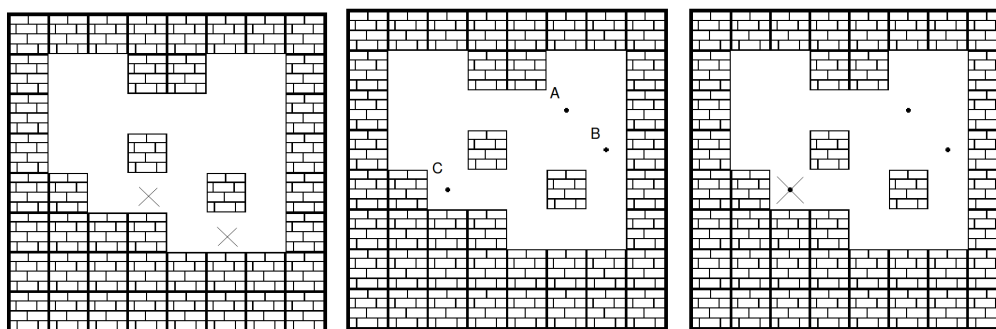


Figure 3.6

# of generation failures		7
# of problems that have no solutions		245
solvable problems	removed	204
	outputted	44
# of trials		500
real good problems		14

Table 3.2: Experimental results of the automatic Sokoban generator [21].



(a) GOAL_AVOID

(b) Setting of goal areas

(c) COR_GOAL

Figure 3.7

Solving Sokoban

Early Attempts at Solving Sokoban

In the late 1990s and 2000s, the focus of Sokoban research shifted from the difficulty of the problem to its solution. Early attempts at solving Sokoban relied on variations of traditional algorithmic search such as A*. Junghanns et al. presented numerous research projects on Sokoban in the late 1990s. In one of these works, they attempted to solve all 90 puzzles of the XSokoban test suite using IDA* [38, 39, 40]. The researchers demonstrated a modified IDA* search method named “Rolling Stone” that incorporates a Deadlock Table to identify dead-end configurations earlier in the heuristic search-tree expansion in order to prune the search. The Deadlock Table consisted of 22 million entries of frequently occurring board patterns that lead to a dead end. Rolling Stone also introduced a concept called “Macro Moves” that collapses a sequence of forced moves into single steps for more efficient memory use during search while preserving solution optimality.

The Rolling Stone algorithm was able to find solutions to 16 of the 90 problems with a search tree shallower than the solution itself. This was made possible by Macro Moves. However, this method comes with significant limitations, as it requires a 22-million-row deadlock pattern database, a piece of hard-coded domain knowledge. The paper also recognizes that Rolling Stone spends 90% of its execution time updating the lower bounds, which means most of the processing time is spent on optimizing a found solution [40].

A similar method was tried by Pereira et al. [41], where the researchers employed a pattern database, a pre-computed lookup table that stores the shortest distances for simplified local problems, to be used as a heuristic in the IDA* search. The authors introduced a concept called instance decomposition. Because it is not possible to know which stone is placed on which goal position before the puzzle is solved, instance decomposition assigns each stone to a specific goal before running the search. This constraint acts as an

intermediate goal and converts the multi-goal problem into a single-goal problem. During the search, the Pattern Database provides an admissible heuristic to be used for pruning the search tree. This motivates the search for better heuristics and representations.

Reinforcement Learning Approaches

Pattern-based search methods remained the dominant approach to Sokoban until Google released I2A, an Imagination-Augmented Agent based on reinforcement learning.

I2A and Boxoban

In 2017, Google researchers introduced a novel architecture [7] for reinforcement learning (RL) that bridges the gap between model-free and model-based approaches. Unlike standard agents that react directly to the environment, Imagination-Augmented Agents (I2As) learn to “imagine” future steps using an internal, potentially imperfect environment model. The agent uses the environment model to sample several future trajectories to inform its next move.

Classical planning methods, such as Monte Carlo Tree Search (MCTS), expand search branches based on a model’s predictions, which typically requires the model to be perfectly accurate. I2As circumvent this limitation by using an encoder to extract meaningful features from simulated trajectories, making the system significantly more robust to model inaccuracies or failures.

The I2A architecture consists of three major components.

First is the Imagination Core (IC). The IC is a type of model that is now more commonly known as a World Model [42]. This model is pretrained on trajectories of the environment. The transition model M takes the current observation s_t and a proposed action a_t to predict the subsequent state s_{t+1} and the immediate reward r_{t+1} :

$$M(s_{t+1}, r_{t+1} \mid s_t, a_t)$$

Following the IC is the Rollout Strategy. To generate an imagined trajectory, the

agent must decide not just the first action, but a sequence of future actions. The rollout strategy is typically a simplified, distilled version of the agent’s current policy. It performs n look-ahead steps to produce a rollout consisting of imagined states, actions, and rewards. This component is essential because it determines which parts of the future state-space the agent explores during its deliberation phase.

The third component is the Rollout Encoder. Traditional model-based RL often fails if the internal model is inaccurate, as the agent might trust a high-reward path that does not actually exist. The Rollout Encoder processes the entire sequence of imagined transitions and compresses them into a fixed-dimensional vector. The encoder learns to extract features from the imagined trajectory. If the IC is inaccurate, the encoder can learn to identify those inconsistencies.

Finally, the Aggregator is a network that uses the current environment state and the imagined path from the rollout encoder to select an action. By replacing parts of MCTS with pretrained networks, I2A can perform tasks such as solving Sokoban puzzles.

Google demonstrated the efficacy of the I2A agent using Boxoban, the same procedurally generated 10×10 Sokoban dataset described in the introduction. The performance of I2A was compared against prominent model-free architectures, specifically A3C (Asynchronous Advantage Actor-Critic). While the A3C agent achieved a 60% solve rate, the I2A agent solved 85% of the same test set. Furthermore, I2A proved more resource-efficient; because its training procedure is partially independent of the live environment, it required substantially fewer environment interactions to converge.

Forward-Backward Reinforcement Learning

Shoham et al. [43] addressed the sparse reward problem in Sokoban by training two complementary networks: a backward-facing model and a forward-facing model. Because reward in Sokoban is only received when all boxes are on goal positions, the backward model is initialized at the goal state and trained on a reversed task by pulling boxes away from goals rather than pushing them towards them. This reversed task provides

dense reward early in training and allows the agent to learn a backward value function V_B efficiently. From this function, a backward trajectory T is constructed for each puzzle instance by initializing at the goal state and using a planning algorithm to find the path that maximizes V_B .

The forward-facing value function V_F then conditions on both the current state s and instance-specific hint features derived from the backward trajectory T . This conditioning partially alleviates the sparsity of the forward reward signal by providing structural information about where boxes should ultimately be placed. The authors note that despite this novel approach, classical IDA* search still provides the strongest overall performance among the methods compared, underscoring the continued relevance of principled heuristic search for Sokoban.

Learning Generalized Reactive Policies

Although I2A achieves strong performance, reaching its best results requires a large amount of data: both to determine the best action and to train a world model of the environment itself. This means that the bottleneck of this architecture is the computation required to train the world model. Groshev et al. [6] proposed a more computationally efficient method in which a deep neural network learns a Generalized Reactive Policy (GRP) that directly maps a problem instance and the current state to an action. The DNN is then used as a heuristic function in an A* search.

To train the DNN, the authors first generated a dataset of successful solution trajectories for a set of Sokoban levels. Trajectories are produced by the Fast-Forward (FF) planner [44], a domain-independent heuristic search planner that drives an Enforced Hill-Climbing search, (a type of greedy local search) using a non-interacting planning heuristic that estimates cost as if subgoals never interfered with each other (i.e., boxes do not block each other). FF may fail to find a solution even when one exists, in which case it falls back to a best-first BFS variant. Each plan returned by FF is a sequence $(s_0, a_0, s_1, a_1, \dots, s_T)$ in which s_0 is the level’s initial state, s_T is a goal state, and each a_i is the action FF chose

at s_i . From every plan, a supervised dataset is extracted by pairing each intermediate state s_i with two labels: the action a_i that FF took next, and the remaining plan length $T - i$. The DNN is then trained in fully supervised fashion against these two targets, so the BFS-grounded FF search acts purely as an offline label generator.

The core contribution of this work is the demonstration that a DNN can learn a heuristic function for A* search based on solution trajectories. Unlike prior approaches that relied heavily on hand-crafted domain knowledge or manually engineered feature representations, their method trains the policy from scratch using only a set of successful execution traces, reducing dependence on problem-specific expertise. For Sokoban, the authors employed a multi-headed convolutional neural network that jointly predicts both action probability and plan length from the current board state. The success of this architecture on spatial planning tasks provides motivation for the use of convolutional processing as a compression baseline in our own approach.

DeepCubeA

DeepCubeA, proposed by Agostinelli et al. [8], replaces the hand-crafted heuristic in A* with a deep neural network h_θ trained to estimate the cost-to-go from any state to the goal. The network is trained through a self-supervised procedure called *Autodidactic Iteration*: beginning from the solved (goal) state, random sequences of actions of increasing length are applied to generate scrambled states. At each depth d , the network learns to predict d as the cost-to-go from the resulting state. This backward induction scheme produces a heuristic that approximates true distance across the reachable state space without any hand-engineered domain knowledge.

At inference time, DeepCubeA uses Weighted A* (WA*), scaling the learned heuristic by a weight $w \geq 1$:

$$f(n) = g(n) + w \cdot h_\theta(n)$$

A weight $w > 1$ biases node expansion toward the goal, sacrificing the optimality guar-

antee in exchange for significantly faster convergence, a practical trade-off for large state spaces where optimal search is intractable. DeepCubeA demonstrated strong performance on challenging combinatorial puzzles including the Rubik’s Cube and the 15-puzzle, establishing it as a general-purpose learned search framework. Subsequent work extending this approach, which uses Answer Set Programming to specify goals to deep neural networks, demonstrated the framework’s applicability to Sokoban, successfully solving highly difficult instances involving more than 20 boxes [9].

Methodology

Boxoban

All levels of the Boxoban puzzle set are constrained to a 10×10 grid, providing a standardized yet vast state space for evaluating an agent’s ability to generalize across millions of unique configurations. Boxoban is partitioned by Google into *unfiltered*, *medium*, and *hard* difficulty splits; this thesis draws every puzzle, both training and evaluation, from the *medium* split, so both distributions are matched in difficulty.

From the medium split, 14,000 puzzles out of the full 9 million corpus were sampled and solved with the pruned BFS described later in this chapter to form the training set, and a disjoint set of 294 puzzles from the same medium split was held out as the evaluation set. A CNN data-scaling sweep (Figure 6.2) shows that solve rate plateaus once the training set reaches a few thousand levels, so 14,000 was chosen as a point well past the plateau while remaining tractable to solve with BFS within available compute.

Heuristic Approaches

Four heuristics are compared in this thesis: a non-learned Manhattan-distance heuristic that serves as the minimum baseline, the CNN cost-to-go network adapted from DeepCubeA that serves as the upper-limit baseline, and two neuroevolved heuristics (NEAT and CoDeepNEAT) whose performance is measured against both bounds. The learned heuristics share the same input representation and evaluation process. The input to each learned heuristic is the flattened multi-hot grid state $s_t \in \{0, 1\}^{500}$, a $(5 \times 10 \times 10)$ binary tensor encoding walls, boxes, goals, and the player. Each heuristic outputs a scalar estimate $\hat{h}(s_t)$ of the remaining steps to solution. Node expansion during A* search is prioritized by:

$$f(s_t) = g(s_t) + \hat{h}(s_t)$$

where $g(s_t)$ is the cost incurred from the start. This preserves optimality when the heuristic is admissible and isolates differences in solve rate and expansion count to heuristic quality rather than search-time inflation. The four approaches differ in how $\hat{h}$ is parameterized and optimized. All three learned heuristics are trained against the same supervised cost-to-go labels, the construction of which is described in the following section.

Manhattan Distance (Minimum Baseline)

The Manhattan-distance heuristic is the classical non-learned cost-to-go estimate for Sokoban-like push puzzles. For a state s_t , it sums over every box the minimum Manhattan distance from that box to any goal cell:

$$\hat{h}_{\text{Manhattan}}(s_t) = \sum_{b \in \text{boxes}(s_t)} \min_{g \in \text{goals}} (|r_b - r_g| + |c_b - c_g|)$$

Boxes already on a goal contribute zero. The heuristic is admissible under unit-cost pushes because pushing a box from (r_1, c_1) to (r_2, c_2) requires at least $|r_1 - r_2| + |c_1 - c_2|$ push moves, regardless of walls, deadlocks, or interactions between boxes. It carries no learned parameters, no domain-specific deadlock knowledge, and no knowledge of the player’s position; it serves as the *minimum* baseline that any learned heuristic must beat in order to justify the cost of training.

CNN Baseline (Upper-Limit Baseline)

The CNN baseline is a fixed-topology convolutional network trained by gradient descent (AdamW with cosine learning-rate schedule [45] and an asymmetric Huber loss that penalizes overestimation more heavily to bias the heuristic toward admissibility). Following the cost-to-go formulation of DeepCubeA [8, 9], this network serves as the upper-limit reference against which the parameter-efficient neuroevolved heuristics are compared. The

input is reshaped to $(10 \times 10 \times 5)$ and processed by four convolutional layers followed by three dense layers:

Layer	Configuration	Parameters
Conv1	64 filters, 3×3 , stride 1	2,944
Conv2	64 filters, 3×3 , stride 1	36,928
Conv3	128 filters, 3×3 , stride 2 ($10 \times 10 \rightarrow 5 \times 5$)	73,856
Conv4	128 filters, 3×3 , stride 1	147,584
Dense	$3,200 \rightarrow 512$	1,638,912
Dense	$512 \rightarrow 256$	131,328
Dense	$256 \rightarrow 1$	257
Total		2,031,809

Table 5.1: Layer-wise parameter counts for the CNN baseline heuristic.

The dominant cost is the first dense layer following the convolutional stack, which connects 3,200 flattened spatial features to 512 hidden units.

NEAT

The NEAT heuristic evolves both the weights and topology of a feed-forward network from scratch. The genome is initialized with 500 input nodes, 1 output node, zero hidden nodes, and a 5% sparse partial direct connection from inputs to the output, which is approximately 25 connections at initialization. Each genome is evaluated against the full training set every generation, computing fitness as the negative mean absolute error between the predicted cost-to-go $\hat{c}_i$ and the ground-truth label c_i over all N training states:

$$\text{fitness} = -\frac{1}{N} \sum_{i=1}^N |\hat{c}_i - c_i|$$

This matches the data exposure of the CNN baseline, which also processes the entire training set each epoch, ensuring that any accuracy difference between the two approaches reflects model capacity rather than data access. Structural mutations (add-node, add-connection) grow the network only where doing so improves fitness, and speciation protects innovations from being discarded before their weights can be refined.

The initial genome has approximately **26 parameters** (25 connection weights and 1

output bias). The winner genome after evolution is expected to remain in the range of tens to a few hundred parameters, which is several orders of magnitude smaller than the CNN baseline.

CoDeepNEAT

CoDeepNEAT coevolves two NEAT populations: *modules* (individual dense-dropout blocks) and *blueprints* (topology graphs whose nodes point to module species). During each generation, 10 blueprints are assembled with 3 genomes per blueprint against a module population of 50, yielding 30 complete Keras models per generation. Each assembled genome is trained with Adam on an 8,192-state subsample of the training set for 5 gradient-descent epochs (batch size 512) using MAE as the loss, then scored on a held-out validation split. Fitness is assigned as $1/(1 + \text{val_MAE})$ and attributed back to both the blueprint and all modules that participated in the assembled genome, so that structural components which consistently contribute to low-MAE assemblies accumulate reproductive advantage across generations. Evolution runs for 50 generations.

This interleaving means the gradient-descent stage does not train any single network to convergence; instead, 5 epochs on a resampled subset provide a noisy but inexpensive estimate of how well a given topology can fit the data, and the evolutionary loop uses that estimate to select topologies that are both trainable and accurate. The subsample refreshes on each genome evaluation, which prevents any one architecture from overfitting to a fixed slice and keeps the fitness signal comparable across the population. Across the 50-generation run, the model with the lowest validation MAE is checkpointed with its trained weights and used directly at A* evaluation time; no separate full-dataset retraining stage is applied.

The module search space constrains each Dense layer to between 64 and 512 units (step 64), with an optional dropout layer (rate 0–0.3). Blueprint mutation can add or remove nodes, so the depth of the assembled network grows through evolution. Although CoDeepNEAT in principle supports convolutional, fully connected, and recurrent layer

types per node [11], the module space in this thesis is restricted to dense-dropout blocks so that the evolved architecture differs from the NEAT genome only in its weight optimizer (Adam gradient descent versus Gaussian weight perturbation), isolating the contribution of gradient information to the comparison. The CNN baseline’s 100% solve rate motivates extending the module space to convolutional blocks in future work, as discussed in the conclusion. Representative parameter counts for plausible evolved topologies are:

Architecture	Parameters
$500 \rightarrow 64 \rightarrow 1$	32,129
$500 \rightarrow 128 \rightarrow 1$	64,257
$500 \rightarrow 128 \rightarrow 64 \rightarrow 1$	72,449
$500 \rightarrow 256 \rightarrow 1$	128,513
$500 \rightarrow 256 \rightarrow 128 \rightarrow 1$	161,281
$500 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 1$	169,473

The table above lists illustrative architectures within the per-layer search space rather than upper bounds on the full assembled network: because blueprint mutation can stack multiple modules and unit counts up to 512 are permitted, deeper or wider assemblies are reachable during evolution. The actual evolved winner is a $500 \rightarrow 320 \rightarrow 320 \rightarrow 1$ network with 263,361 trainable parameters, roughly $7.7\times$ smaller than the CNN baseline.

Forward BFS Trajectories and Cost-to-Go Labels

Training labels for the three learned heuristics are derived directly from forward breadth-first search (BFS) solutions. For each Boxoban puzzle, BFS expands states outward from the initial configuration until it reaches a goal state, producing a shortest-path trajectory $s_0, s_1, \dots, s_T$ where s_0 is the given initial state and s_T is the first goal state encountered. During the labeling pass, each state s_i on this trajectory is assigned a cost-to-go label $c_i = T - i$, where i is the step index along the trajectory.

A standard BFS applied to the full Sokoban state space is computationally intractable

because many branches lead to deadlock configurations that can never reach a solution, causing the search frontier to grow unnecessarily large. To make forward BFS feasible, three pruning mechanisms are applied during expansion, each one cheap enough to evaluate inline on every successor:

- **Dead-square precomputation.** Before search begins, a backward reachability pass is run from every goal cell: a square is marked *live* if there exists some box position and matching player position from which a box could be pushed onto it and then on to a goal, and every remaining non-wall square is marked *dead*. Any successor state that places a box on a dead square is immediately pruned, because no sequence of legal pushes from that configuration can deliver that box to a goal. This is strictly stronger than corner-only detection, since it also catches wall-hugging squares with no goal along the wall segment.
- **Frozen 2×2 deadlock detection.** After every box-pushing move, the successor state is scanned for any 2×2 block in which all four cells are either walls or boxes. If such a block contains at least one box not on a goal, the configuration is a permanent deadlock (none of those boxes can ever move again) and the branch is pruned. This check is only invoked when the most recent action actually pushed a box, so it adds no cost to pure player-movement transitions.
- **Reverse-move pruning.** If the last action was a player move that did *not* push a box, the immediately opposite action is forbidden from the successor set. This removes the trivial back-and-forth symmetry in which the agent walks one step and undoes it, without affecting solution completeness (any state reachable via a reverse move is also reachable by the preceding state with that move omitted).

In addition to these pruning rules, BFS deduplicates by a state hash over the player position and box positions, so identical configurations reached by different action sequences are expanded at most once. Together these mechanisms keep the frontier small enough that medium-split puzzles solve within a half-million-node budget.

Because BFS returns a shortest path under uniform step cost, these labels are the exact ground-truth cost-to-go for every state on the trajectory: the goal state s_T receives label 0, and the initial state receives label T .

The resulting search produces compact, provably correct solution trajectories that serve as the supervised signal for training the CNN, NEAT, and CoDeepNEAT heuristics. The idea of training a heuristic on exact cost-to-go labels derived from solved trajectories is adapted from the Autodidactic Iteration procedure of DeepCubeA [8] and its Sokoban extension [9]. The key distinction from those works is the direction of trajectory generation: DeepCubeA constructs training states by applying random reverse moves starting from the goal, whereas this thesis obtains trajectories by forward-solving each puzzle with pruned BFS and reading labels off the solved path. Forward-solving guarantees that every training state is reachable from a real Boxoban initial configuration along a shortest path, which matches the state distribution the heuristic will encounter at A* evaluation time. All three learned heuristics are trained on the same (state, cost-to-go) pairs, so any performance difference reflects model capacity rather than differences in data access or quality.

Training Metrics

The CNN was trained by minimizing the Huber loss [46] on batches of (state, cost-to-go) pairs, with overestimation penalized at $2\times$ to bias toward admissibility.

NEAT and CoDeepNEAT use a fitness function to train the network either fully or partially. The fitness function of NEAT is $-\text{MAE}$, and the best genome by validation MAE is checkpointed.

CoDeepNEAT uses MAE as the loss function for gradient descent and $1/(1+\text{val_MAE})$ as the fitness function for evolving the network architecture with NEAT.



Figure 5.1: Visualization of a BFS solution trajectory for a single Boxoban level. The blue square is the player, and the brown boxes are the boxes that need to be pushed onto red goal positions. Green squares are boxes already on goal positions. Each frame shows a state along the path from initial configuration to goal; the cost-to-go label assigned to that state is the number of remaining frames.

Results

Experiments and Results

All four heuristics were evaluated on 294 held-out levels from the Boxoban medium set under A* search with a maximum node expansion budget of 100,000. The CNN baseline was additionally evaluated across a range of training set sizes to characterize its data scaling behavior. Two reference points bracket the comparison: the Manhattan-distance heuristic anchors the *minimum* baseline that any learned heuristic must beat to justify its training cost, and the CNN cost-to-go network adapted from DeepCubeA anchors the *upper-limit* baseline of what a fully gradient-trained, parameter-rich architecture can achieve on the same inputs. Table 6.1 summarizes the four solve rates side by side.

Heuristic	Parameters	Solve Rate
Manhattan distance (minimum baseline)	0	87.4% (257/294)
NEAT (average across 9-config sweep)	~ 25	83.9% (246.8/294)
CoDeepNEAT	263,361	88.8% (261/294)
DeepCube CNN cost-to-go (upper-limit baseline)	2,031,809	100.0% (294/294)

Table 6.1: Solve rate of each heuristic on the 294 held-out medium-split Boxoban levels under A* search with a 100,000 node expansion budget. The NEAT row reports the mean solve rate across the nine-configuration hyperparameter sweep (Appendix A.4); per-configuration rates ranged from 82.7% to 84.7%. Manhattan distance is the minimum baseline and the DeepCube-style CNN cost-to-go network is the upper-limit baseline.

Manhattan Distance (Minimum Baseline)

The Manhattan-distance heuristic solves 87.4% of the held-out levels (257/294) under the same 100,000 expansion budget as every other method, despite having zero trainable parameters and computing only the per-box L1 distance to the nearest goal cell (Figure 6.1). The A* search itself applies dead-square precomputation, frozen 2×2 deadlock detection, reverse-move suppression, and state-hash deduplication identically to every heuristic

in this thesis, so the 87.4% figure isolates the contribution of the heuristic signal under search-side pruning that is held fixed across all four methods. Because the heuristic is provably admissible under unit-cost pushes, every recovered solution is also optimal-length: the average solution-length to optimal-length ratio is 1.00 across all solved levels.

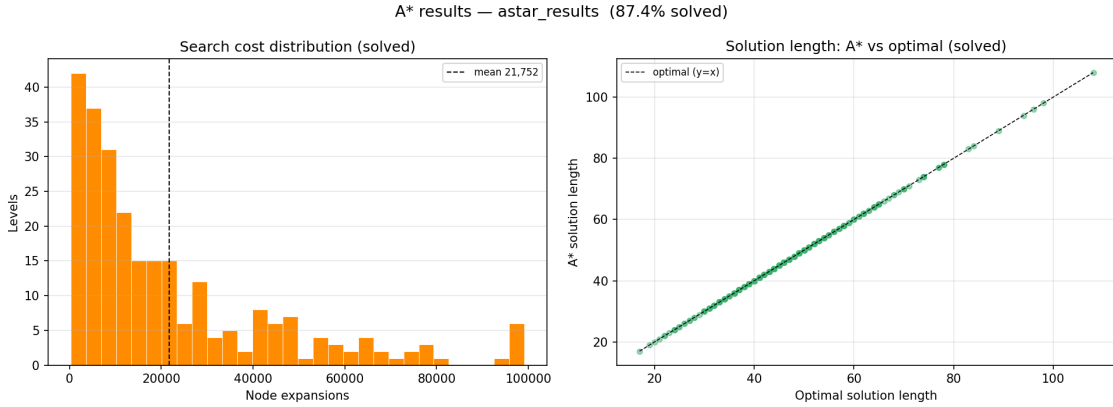


Figure 6.1: Manhattan-distance heuristic A* search results on the 294 held-out medium-split levels under the 100,000 expansion budget. Solve rate: 87.4% (257/294).

A parameter-free heuristic clearing 87% of the medium split establishes the floor that any learned heuristic must beat to justify its training cost. Framing the comparison around this floor yields the central result of the thesis: pure NEAT, despite searching over both topology and weights, fails to clear it, while CoDeepNEAT, which adds gradient-based weight optimization to the evolutionary search, beats it.

CNN Baseline (Upper-Limit Baseline)

The CNN baseline is trained on the same 14,000 BFS-solved puzzles drawn from the Boxoban medium split and evaluated on a disjoint set of 294 puzzles from the same split. With this training set, the CNN achieves a 100% solve rate (294/294) under the 100,000 node expansion budget, which serves as the upper-limit reference for the neuroevolved heuristics. The data-scaling analysis (Figure 6.2) is not a separate baseline but an earlier ablation that swept the training set size under a less refined training configuration; its 14,000-trajectory point reached 98.6% (290/294) rather than 100%. Subsequent runs

with the refined training configuration used elsewhere in this thesis reach the 100% reported above. Both numbers are measured at the same 100,000 expansion budget on the same held-out 294 puzzles, so the gap between them reflects training-side hyperparameter choices, not data quantity or evaluation protocol.

The data-scaling curve (Figure 6.2) shows that solve rate rises steeply with training set size and then converges: 69.7% at 100 trajectories, 89.1% at 500, 94.6% at 1,000, 97.3% at 5,000, 98.0% at 10,000, and 98.6% at 14,000 trajectories under the ablation’s training configuration. The marginal gain from each tenfold increase in training data shrinks rapidly, and the later ablation that retrained at 14,000 trajectories with refined hyperparameters reached the 100% reported above, cementing 14,000 as the effective benchmark training set. Together these results justify using 14,000 trajectories rather than the full nine-million-level Boxoban corpus: the heuristic is not data-limited beyond a few thousand training examples, and additional trajectories yield diminishing returns at substantial BFS-solving cost.

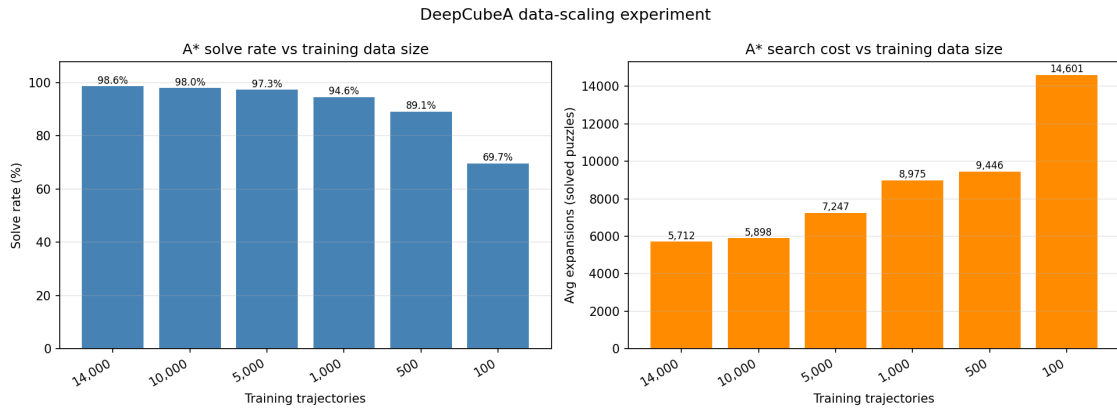


Figure 6.2: CNN baseline solve rate as a function of training set size.

The 100,000 expansion budget is imposed deliberately so that the CNN, NEAT, and CoDeepNEAT heuristics are compared under identical search-time budgets. A less accurate heuristic will explore more nodes per level to reach a solution, and without a fixed cap the comparison would conflate heuristic quality with total search cost. The cap was chosen so that the CNN baseline could solve essentially every level (it reaches 100% at

this budget), while still meaningfully constraining the neuroevolved heuristics. Reporting all three methods under the same cap therefore isolates how heuristic accuracy translates into search efficiency.

NEAT Discussion

The NEAT heuristic solves 84.0% of the 294 held-out levels (247/294, Figure 6.3; full hyperparameter sweep in Appendix A.4), leaving 47 unsolved. This is 3.4 percentage points *below* the parameter-free Manhattan-distance baseline (87.4%, Figure 6.1), so on this benchmark pure NEAT fails to clear the minimum baseline that any learned heuristic must beat to justify its training cost.

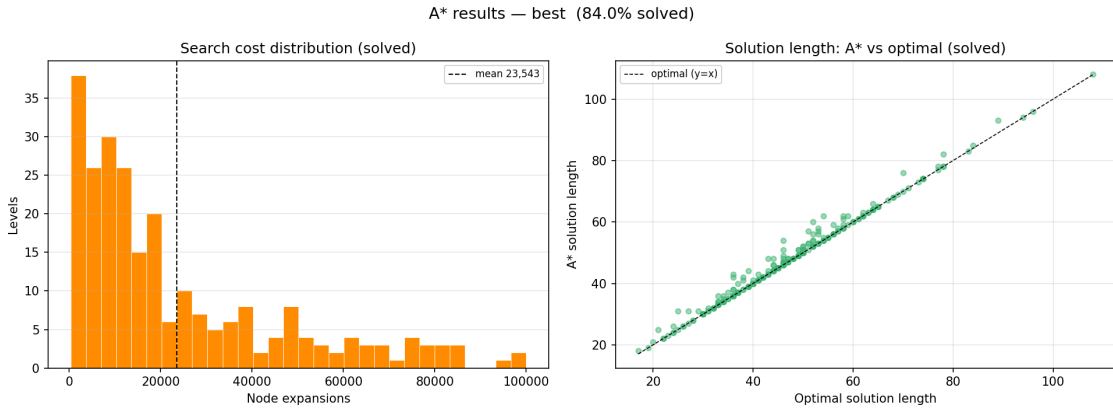


Figure 6.3: NEAT baseline (`conn_del=0.5`) A* search results on the 294 held-out medium-split levels under the 100,000 expansion budget. Solve rate: 84.0% (247/294).

The winner genome contains 4 internal nodes and 21 enabled connections for a total of 25 trainable parameters. This is approximately 0.001% of the 2,031,809-parameter CNN baseline. Figure 6.4 visualizes the genome by mapping every enabled input-to-network connection back to its position in the $5 \times 10 \times 10$ input tensor: 12 of the 21 enabled connections feed directly into the output node, while the remaining 9 are split evenly across three hidden nodes (two input edges plus one hidden-to-output edge each). The connectivity is heavily biased toward the BOX channel, with secondary contributions from WALL, GOAL, and PLAYER, and only sparse use of the BG channel.

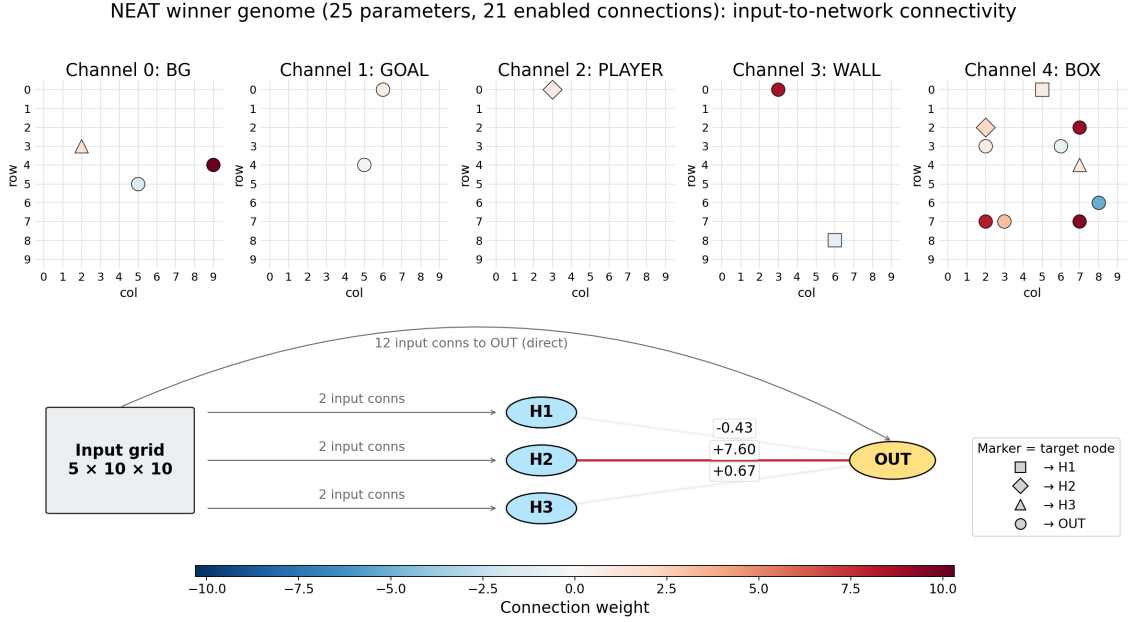


Figure 6.4: NEAT winner genome topology. Each scatter point on the five 10×10 channel grids marks a single enabled connection from an input cell into the network; marker shape indicates the target node ($\bigcirc$ output, $\square/\diamond/\triangle$ hidden nodes H1/H2/H3) and color encodes connection weight (red positive, blue negative). The lower panel shows the four-node internal topology with each hidden node’s three-input fan-in summarized and the three hidden-to-output weights shown explicitly.

The topology in Figure 6.4 is the most direct evidence for why pure NEAT fails to clear the Manhattan floor. Out of $5 \times 10 \times 10 = 500$ candidate input cells, only 21 enabled connections survive, and the connectivity is heavily concentrated on the BOX channel. Concentrating fan-in on BOX is, on its own, a meaningful inductive bias: the cost-to-go of a Sokoban state is dominated by which boxes are placed where, so a heuristic that reads box positions strongly is at least pointing at the right input. The decisive failure is what is missing alongside it. The Manhattan heuristic is accurate because it pairs each box position with the nearest goal position; the evolved genome reads box positions but draws only sparse, idiosyncratic taps from the GOAL channel and almost nothing from PLAYER, so it has no way to compute even a rough analogue of per-box distance-to-goal. Without GOAL coupling, BOX-channel weights collapse into a position-specific look-up: certain board regions get rewarded as “probably high cost” and others as “probably low cost” regardless of where the goals actually are on that puzzle.

That collapse is exactly what the negative-MAE fitness function rewards on the training distribution. Long-solution puzzles, those with high ground-truth cost-to-go, contribute disproportionately to absolute error: a single 70-step puzzle mispredicted by 20 hurts the fitness more than ten 25-step puzzles each mispredicted by 2. The survival pressure on the genome is therefore biased toward fitting the heaviest-cost puzzles in the training set, and the cheapest way to fit them with a 25-parameter network is to memorize spatial cues from box positions that co-occur with long solutions. The resulting heuristic is not a generalizable cost-to-go estimator; it is a small position-keyed regression onto the high-cost tail of the training distribution. On unseen puzzles where the box positions do not match those memorized patterns, the heuristic provides no information beyond what the BOX-only inputs accidentally imply, and A* search falls back to expanding broadly. This explains both why solve rate falls below Manhattan (the analytical heuristic uses the GOAL channel that NEAT effectively ignored) and why solve rates are nearly uniform across the NEAT hyperparameter sweep: when the heuristic carries little information beyond a position-keyed prior, the search is led primarily by the shared search-side pruning machinery (dead-square precomputation, frozen 2×2 deadlock detection, reverse-move suppression, and state-hash deduplication) rather than by the heuristic itself, so swapping NEAT hyperparameters moves solve rate by only a fraction of a percentage point.

The deeper cause is granularity of the optimizer. NEAT adjusts weights only through Gaussian perturbation under the `weight_mutate_power` schedule, and the hyperparameter sweep in Appendix A.4 confirms that varying this perturbation magnitude between 0.3 (`wt_fine`) and 0.8 (`wt_coarse`) only moves solve rate by a fraction of a percentage point and never crosses the Manhattan baseline. The weight signal that a heuristic on 500-dimensional binary inputs needs is finer than what coordinate-wise random perturbation, even at small step sizes, can reliably produce: a single useful weight update is too high-dimensional an event to be reached by mutation, and the speciation that protects topology innovations also protects weight configurations that have not yet been refined. Pure evolutionary search is therefore the wrong tool for the weight-tuning half of the

problem on this input dimensionality.

The CoDeepNEAT result is the empirical confirmation of this diagnosis. CoDeepNEAT keeps NEAT’s evolutionary architecture search but replaces weight perturbation with Adam gradient descent on each assembled genome, and that single substitution moves solve rate from 83.9% (below Manhattan) to 88.8% (above Manhattan). The substitution replaces isotropic Gaussian weight perturbation with Adam steps driven by the actual MAE gradient, which is a coordinate-aligned signal rather than a random one. The 4.9 percentage-point jump from pure NEAT to CoDeepNEAT, and the simultaneous crossing of the Manhattan baseline, is therefore attributable to gradient information rather than topology innovation. This is consistent with the topology analysis in Figure 6.4, which identifies weight-perturbation granularity, rather than topological expressiveness, as the binding constraint on pure NEAT.

CoDeepNEAT Discussion

CoDeepNEAT produces a higher solve rate of 88.8% (261/294) than pure NEAT’s 84.0%, and crucially also clears the Manhattan-distance baseline by 1.4 percentage points. CoDeepNEAT is therefore the only neuroevolved method in this thesis that beats the parameter-free analytical baseline; the gradient-descent phase that distinguishes it from pure NEAT is what closes the gap. The evolved best architecture is a $500 \rightarrow 320 \rightarrow 320 \rightarrow 1$ feed-forward network containing 263,361 trainable parameters, about $7.7\times$ smaller than the CNN baseline but roughly $10,500\times$ larger than the NEAT genome. The improvement over pure NEAT is likely driven by the fine-tuned weights produced by the Adam optimizer during each genome’s gradient-descent phase, which captures finer gradient information about the cost-to-go space than NEAT’s pure weight perturbation can. However, this improvement requires roughly $10,500\times$ more parameters than the NEAT genome, yielding only a 4.8 percentage-point increase in solve rate over NEAT and a 1.4 percentage-point increase over the parameter-free Manhattan baseline.

Interpretation of Findings

The split between NEAT and CoDeepNEAT, summarized in Table 6.1, is the central empirical finding. Pure NEAT’s weight-perturbation search is overmatched by the dimensionality of the 500-binary input grid: a 25-parameter genome cannot extract a heuristic signal more accurate than the per-box minimum-Manhattan-distance summation that the Manhattan baseline already provides analytically. CoDeepNEAT, by contrast, clears that baseline. The contrast establishes the central claim of the thesis: neuroevolution *can* beat the Manhattan-distance baseline on Boxoban, but only when architectural search is paired with gradient-based weight optimization; pure topology-and-weight evolution from scratch is not enough on inputs of this dimensionality. The remaining gap to the CNN is non-trivial, but the fact that a $7.7\times$ parameter reduction still clears the Manhattan floor suggests that heuristic compression through neuroevolution is a direction warranting further study.

Conclusion

This thesis asked whether neuroevolution can produce cost-to-go heuristic functions for A* search on Boxoban that are substantially more parameter-efficient than a conventional gradient-trained CNN baseline. The four-way comparison reported in Chapter 6 answers in the affirmative, but only conditionally: pure topology-and-weight evolution from scratch is not enough on inputs of this dimensionality, while pairing evolutionary architecture search with gradient-based weight optimization recovers enough accuracy to clear the analytical baseline at a fraction of the gradient-trained upper bound.

The split between NEAT and CoDeepNEAT is the load-bearing finding. NEAT’s failure to clear the Manhattan floor, despite searching over both topology and weights, is consistent with its known weakness on high-dimensional inputs: weight perturbation cannot supply the gradient information required to outperform a purely geometric estimate on a 500-binary input grid. CoDeepNEAT keeps the evolutionary architecture search but replaces NEAT’s weight perturbation with Adam gradient descent on each assembled genome, and that one change closes the gap. The implication is that the productive locus of neuroevolution research on inputs of this dimensionality is the hybrid: evolution chooses the architecture, gradients tune the weights.

The remaining gap between CoDeepNEAT and the CNN is non-trivial, but the parameter reduction relative to the modest drop in solve rate, and the fact that CoDeepNEAT clears the floor that pure NEAT cannot, suggests that hybrid neuroevolution is a viable strategy for heuristic compression when pure topology evolution is too constrained and a fully hand-designed CNN is too parameter-heavy.

Future Work

The empirical separation between pure NEAT and CoDeepNEAT, on opposite sides of the Manhattan baseline, indicates that further investigation should concentrate on the hybrid method rather than on additional scaling of pure topology-and-weight evolution.

Figure 6.4 indicates that weight perturbation cannot supply the granular updates required by a 500-dimensional binary input, and this is unlikely to be solved by a larger population or longer evolutionary runs alone. The more productive intervention is to replace the weight optimizer, as CoDeepNEAT does. Future work should therefore prioritize CoDeepNEAT-style hybrids in which an evolutionary outer loop searches over architecture while a gradient-based inner loop performs weight tuning, allowing architecture search to proceed without being confounded by stochastic weight perturbation.

Two extensions follow directly. First, the gradient-descent inner loop in CoDeepNEAT trains each assembled genome on a fixed subsample for a fixed number of epochs. Systematic ablation of this inner loop, varying the subsample size, optimizer, loss function, number of epochs, and fitness-aggregation rule across blueprint assemblies, would clarify which components of the gradient signal are responsible for the performance gain over pure NEAT. The same outer-evolution, inner-gradient pattern can also be applied to plain NEAT genomes by interposing a short gradient-descent fine-tuning pass within each fitness evaluation, providing a controlled test of whether gradient information alone is sufficient to raise pure NEAT above the Manhattan baseline, independent of the module/blueprint coevolution machinery.

Second, the upper-limit baseline in this thesis is a convolutional network, and its 100% solve rate constitutes the strongest evidence presented here that convolutional inductive bias underlies the recovery of a generalized cost-to-go estimator on Boxoban’s 10×10 grid. The dense-dropout module space searched by CoDeepNEAT does not provide this prior, which offers a plausible account of why CoDeepNEAT plateaued. Extending CoDeepNEAT’s module space to include convolutional blocks parameterized by filter count, kernel size, stride, and optional pooling, and correspondingly extending NEAT’s primitives

to admit shared-weight convolutional connections in place of purely dense ones, would bring together the three ingredients that distinguish the upper-limit baseline from the neuroevolved methods: spatial inductive bias, gradient-based weight optimization, and parameter-efficient architecture search. Closing the remaining gap to the CNN while preserving the parameter advantage already demonstrated by CoDeepNEAT is the most plausible role for this convolutional-hybrid direction in subsequent work.

Supplementary Results

Manhattan Distance Baseline Results

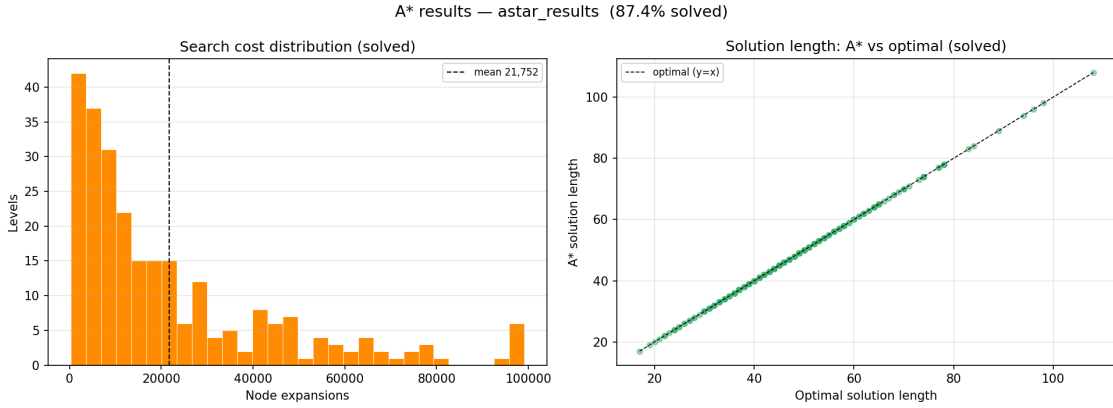


Figure A.1: Manhattan-distance heuristic A* search results on the 294 held-out medium-split levels under the 100,000 expansion budget. Solve rate: 87.4% (257/294). Average solution-length to optimal-length ratio: 1.00 (the heuristic is admissible).

The right panel of Figure A.1 shows that every solved level lies exactly on the $y = x$ optimality line: the A* solution length equals the optimal solution length on all 257 solved puzzles, yielding an average solution-length to optimal-length ratio of 1.00. This is not an empirical accident but a guaranteed consequence of using an admissible heuristic with A* search. The Manhattan-distance heuristic, summing the per-box minimum Manhattan distance to any goal cell, is provably admissible under unit-cost pushes because pushing a box from (r_1, c_1) to (r_2, c_2) requires at least $|r_1 - r_2| + |c_1 - c_2|$ push moves regardless of walls, deadlocks, or interactions between boxes. A* search with an admissible heuristic is itself guaranteed to return optimal-cost solutions whenever it terminates successfully, so a 1.00 length ratio across every solved level is the expected behavior of this configuration rather than an indication that the heuristic is particularly accurate. Heuristic accuracy in this setting is reflected only in the left panel, the search-cost (node-expansion) distribution and

the resulting solve rate under the 100,000-expansion budget; optimality of the recovered solutions is fixed by the admissibility of the heuristic and is not a free parameter of the comparison.

CNN Baseline Results

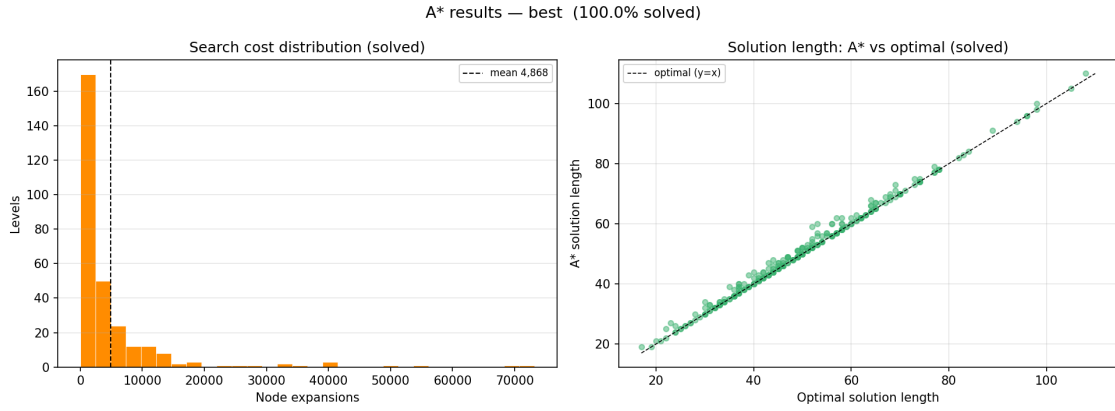


Figure A.2: CNN baseline trained on 14,000 medium-split Boxoban levels: A* search results on the 294 held-out medium-split levels. Solve rate: 100% (294/294) within the 100,000 node expansion budget.

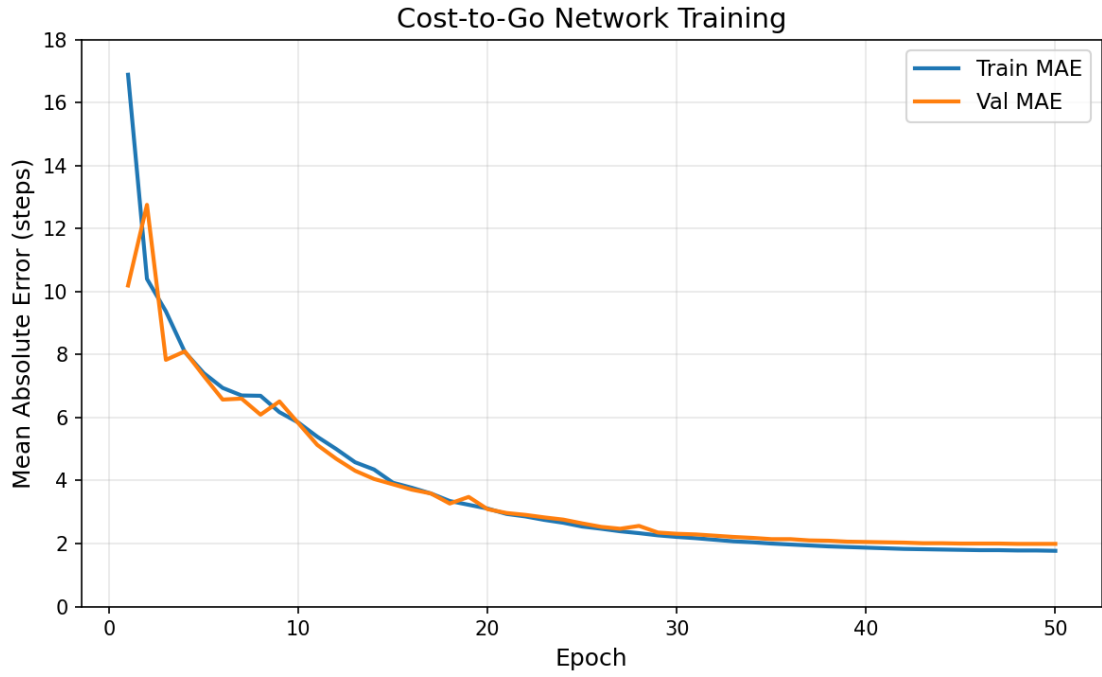


Figure A.3: CNN baseline training curve: validation MAE over training epochs.

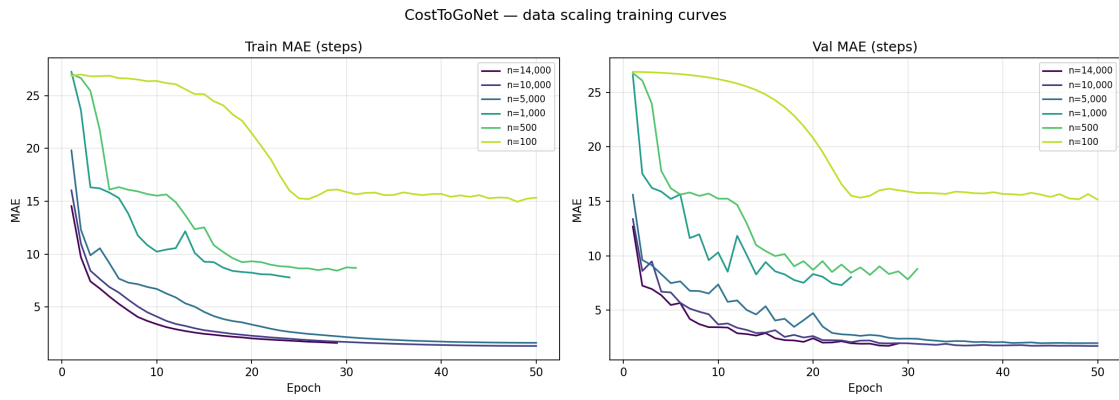


Figure A.4: CNN baseline training curves across different training set sizes.

CoDeepNEAT Results

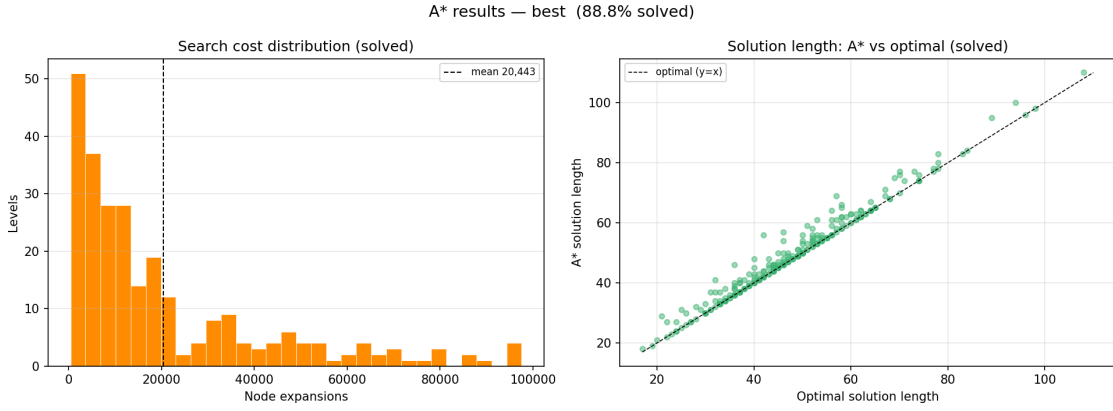


Figure A.5: CoDeepNEAT best genome: A* search results on held-out Boxoban levels. Solve rate: 88.8% (261/294).

NEAT Hyperparameter Sweep

Table A.1 summarizes the NEAT ablation sweep, which varied connection deletion probability, node mutation rates, stagnation limit, and weight mutation power across nine configurations. All configurations used a population size of 150.

Config	conn_del	node_add/del	stagnation	wt_power	Solve Rate
baseline	0.5	0.2 / 0.2	15	0.5	84.0%
prune_light	0.3	0.2 / 0.2	15	0.5	84.7%
prune_heavy	0.7	0.2 / 0.2	15	0.5	84.0%
node_high	0.5	0.4 / 0.4	15	0.5	84.0%
node_low	0.5	0.1 / 0.1	15	0.5	83.0%
stag_25	0.5	0.2 / 0.2	25	0.5	84.0%
stag_40	0.5	0.2 / 0.2	40	0.5	82.7%
wt_fine	0.5	0.2 / 0.2	15	0.3	84.4%
wt_coarse	0.5	0.2 / 0.2	15	0.8	84.7%

Table A.1: NEAT sweep: nine configurations and their A* solve rates.

Per-configuration A* Search Results

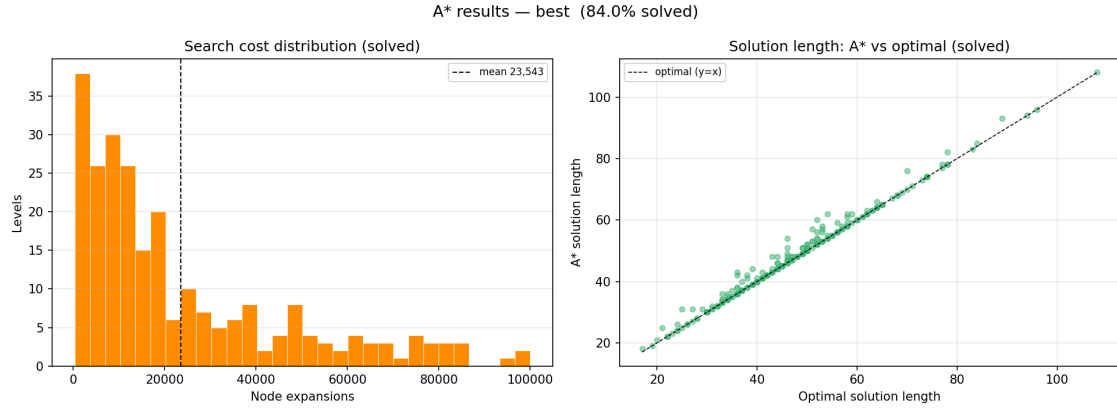


Figure A.6: baseline ($\text{conn_del}=0.5$, $\text{stag}=15$, $\text{wt_pow}=0.5$). Solve rate: 84.0%.

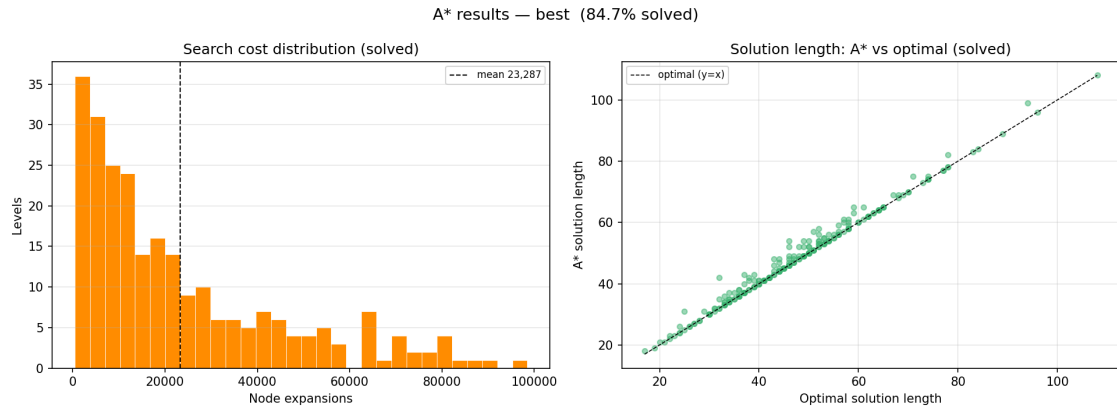


Figure A.7: `prune_light` ($\text{conn_del}=0.3$). Solve rate: 84.7%.

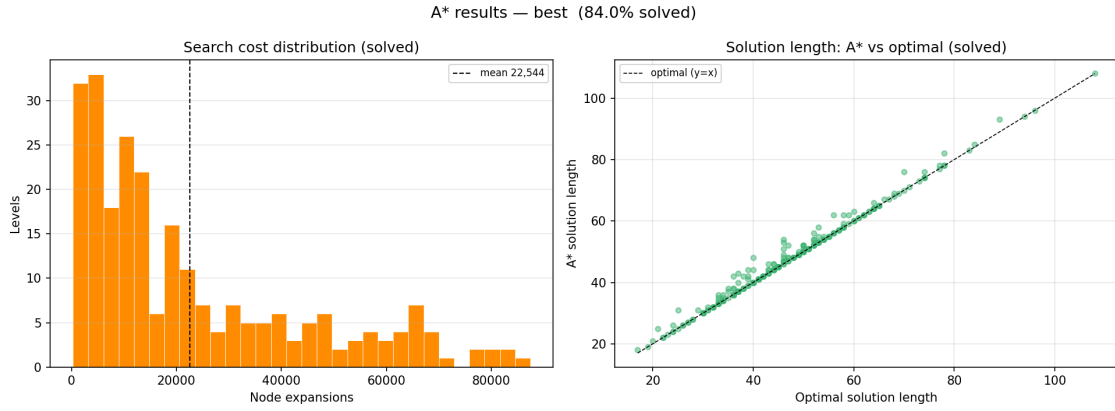


Figure A.8: `prune_heavy` (`conn_del`=0.7). Solve rate: 84.0%.

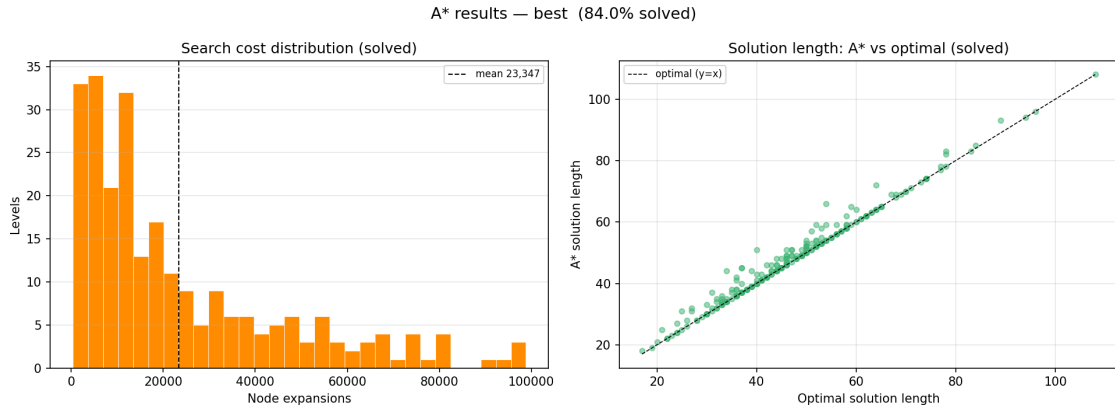


Figure A.9: `node_high` (`node_add/del`=0.4). Solve rate: 84.0%.

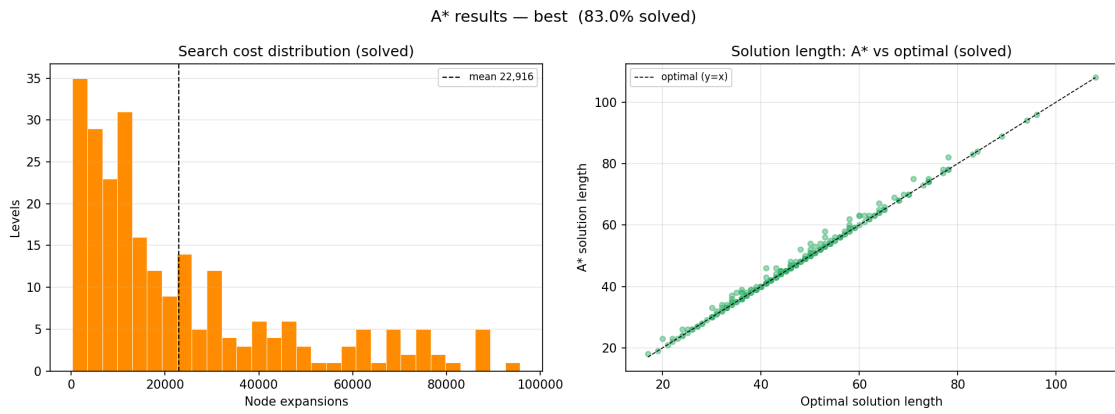


Figure A.10: `node_low` (`node_add/del`=0.1). Solve rate: 83.0%.

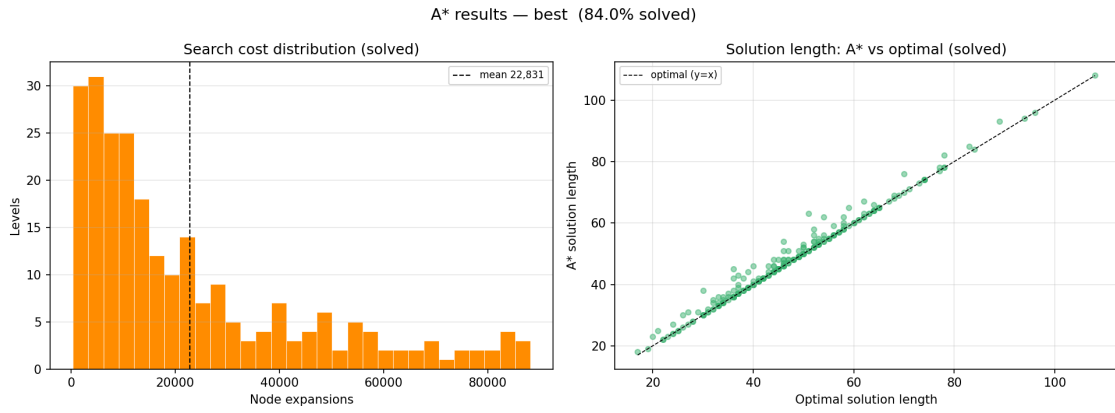


Figure A.11: stag_25 (max_stagnation=25). Solve rate: 84.0%.

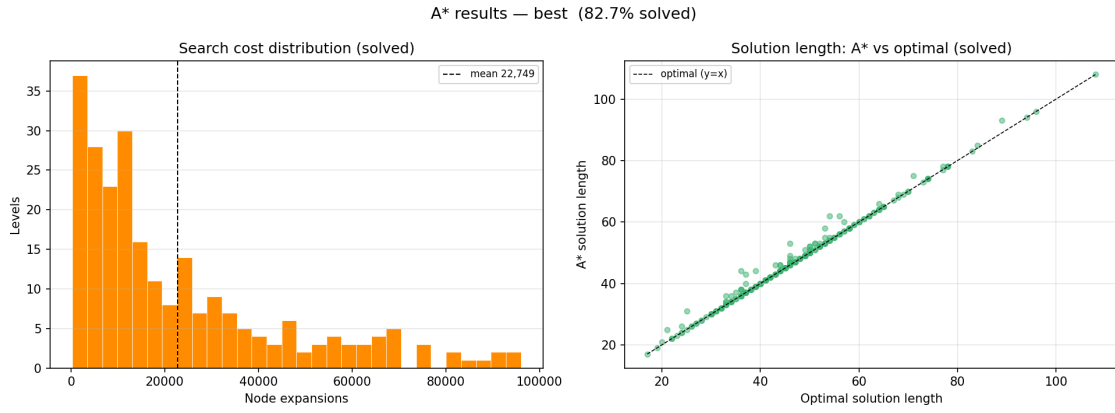


Figure A.12: stag_40 (max_stagnation=40). Solve rate: 82.7%.

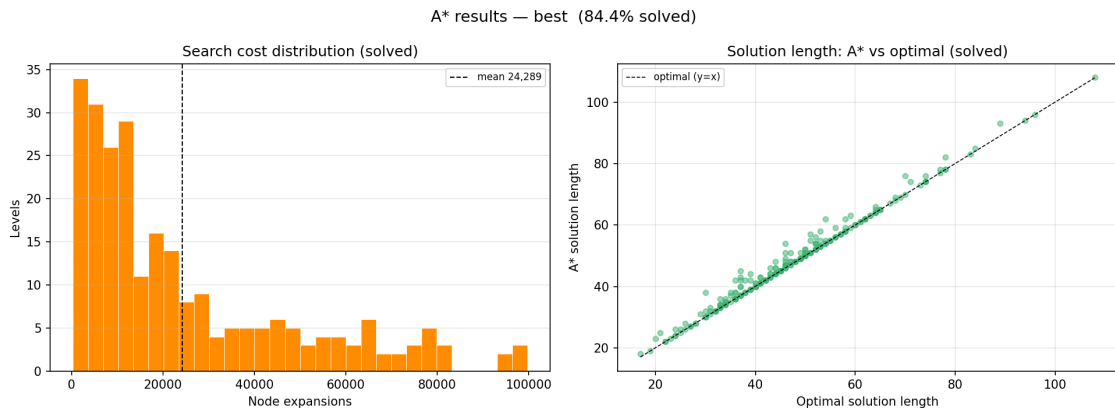


Figure A.13: wt_fine (weight_mutate_power=0.3). Solve rate: 84.4%.

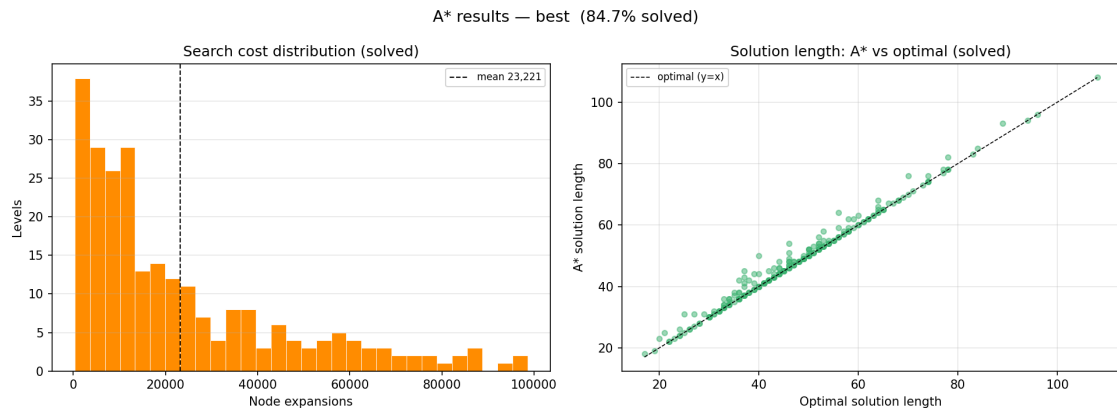


Figure A.14: wt_coarse (weight_mutate_power=0.8). Solve rate: 84.7%.

Bibliography

- [1] Michael Fryers and Michael Greene. “Sokoban”. In: *Eureka* 54 (1995). Archived from the original (PDF) on September 5, 2024, pp. 25–32. URL: <https://www.archim.org.uk/eureka/54/sokoban.pdf>.
- [2] Joseph C. Culberson. *Sokoban is PSPACE-complete*. Tech. rep. TR 97-02. Archived from the original (PDF) on May 28, 2024. Edmonton, AB, Canada: Department of Computing Science, University of Alberta, 1997. URL: <https://webdocs.cs.ualberta.ca/~joe/Preprints/Sokoban/index.html>.
- [3] Brian Damgaard. *YASS: Yet Another Sokoban Solver and Optimizer*. <https://sourceforge.net/projects/sokobanyasc/>. Sokoban solver and optimizer written in Pascal.
- [4] Jorit Wits. *Sokoban++*. <https://github.com/joriswit/YASS>. Sokoban game and solver integrating the YASS plugin.
- [5] Andreas Junghanns and Jonathan Schaeffer. “Domain-dependent single-agent search enhancements”. In: *IJCAI*. 1999, pp. 570–577.
- [6] Edward Groshev et al. “Learning generalized reactive policies using deep neural networks”. In: *Proceedings of the International Conference on Automated Planning and Scheduling*. Vol. 28. 2018, pp. 408–416.
- [7] Sébastien Racanière et al. “Imagination-Augmented Agents for Deep Reinforcement Learning”. In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon et al. Vol. 30. Curran Associates, Inc., 2017. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/7ca66fc0c25b4ca3f707f4ac0d8700ef-Paper.pdf.
- [8] Forest Agostinelli et al. “Solving the Rubik’s Cube with Deep Reinforcement Learning and Search”. In: *Nature Machine Intelligence* 1.8 (2019), pp. 356–363.
- [9] Forest Agostinelli et al. “Obtaining Approximately Admissible Heuristic Functions through Deep Reinforcement Learning and A* Search”. In: *Bridging the Gap between AI Planning and Reinforcement Learning (PRL Workshop, ICAPS)*. 2021.
- [10] Kenneth O. Stanley and Risto Miikkulainen. “Evolving Neural Networks through Augmenting Topologies”. In: *Evolutionary Computation* 10.2 (2002), pp. 99–127.
- [11] Risto Miikkulainen et al. “Evolving Deep Neural Networks”. In: *Artificial Intelligence in the Age of Neural Networks and Brain Computing*. Ed. by Robert Kozma et al. arXiv:1703.00548. Academic Press, 2019, pp. 293–312. DOI: 10.1016/B978-0-12-815480-9.00015-3.
- [12] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. “A Formal Basis for the Heuristic Determination of Minimum Cost Paths”. In: *IEEE Transactions on Systems Science and Cybernetics* 4.2 (1968), pp. 100–107.
- [13] Yann LeCun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.

- [14] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. “ImageNet Classification with Deep Convolutional Neural Networks”. In: *Advances in Neural Information Processing Systems 25*. 2012, pp. 1097–1105.
- [15] Mabrouka Dahouda and Inwhee Joe. “A deep learning-based convolutional neural network for ...” In: *Scientific Reports* 14 (2024).
- [16] Julian Togelius and Simon M Lucas. “Evolving controllers for simulated car racing”. In: *2005 IEEE Congress on Evolutionary Computation*. Vol. 2. IEEE. 2005, pp. 1906–1913.
- [17] Julian Togelius and Simon M Lucas. “Evolving robust and specialized car racing skills”. In: *2006 IEEE International Conference on Evolutionary Computation*. IEEE. 2006, pp. 1187–1194.
- [18] Sebastian Risi and Julian Togelius. “Neuroevolution in games: State of the art and open challenges”. In: *IEEE Transactions on Computational Intelligence and AI in Games* 9.1 (2015), pp. 25–41.
- [19] Giuseppe Cuccu, Julian Togelius, and Philippe Cudré-Mauroux. “Playing Atari with Six Neurons”. In: *Proceedings of the 18th International Conference on Autonomous Agents and MultiAgent Systems*. Montreal, QC, Canada: International Foundation for Autonomous Agents and Multiagent Systems, 2019, pp. 998–1006.
- [20] Andreas Junghanns. “Pushing the Limits: New Developments in Single-Agent Search”. PhD thesis. Edmonton, AB, Canada: University of Alberta, 1999. DOI: 10.7939/R3W95103S. URL: <https://doi.org/10.7939/R3W95103S>.
- [21] Yoshio Murase, Hitoshi Matsubara, and Yuzuru Hiraga. “Automatic Making of Sokoban Problems”. In: *PRICAI ’96: Topics in Artificial Intelligence*. Ed. by Norman Foo and Randy Goebel. Vol. 1114. Lecture Notes in Computer Science. Springer Science & Business Media, 1996, p. 592. ISBN: 978-3-540-61532-3.
- [22] Falcon Co., Ltd. 倉庫番とは [What is Sokoban]. Japanese. Archived from the original on January 28, 2026. Sokoban.jp. 2026. URL: <https://www.sokoban.jp/> (visited on 02/09/2026).
- [23] Obunsha. “PART 2 脳みそを酷使するのである [Part 2: You Will Use Your Brain to the Extreme]”. In: *MICOMGAMES (in Japanese)* 1.1 (Dec. 1983), p. 38.
- [24] ASCII Corporation. “考えるウサギはパソコンの野を駆ける! [The thinking rabbit runs through the fields of the computer!]” In: *LOGiN (in Japanese)* (Dec. 1983), pp. 136–139.
- [25] “Aldebaran”. In: *Monthly Mycom [ja] (in Japanese)* (Jan. 1980). Specific author not listed, pp. 22–28.
- [26] Carlos Montiers Aguilera. *My conversation with Mr Hiroyuki Imabayashi*. English. Archived from the original on February 9, 2026. Sokoboxes. 2020. URL: <https://sokoboxes.com/articles/my-conversation-with-mr-hiroyuki-imabayashi/> (visited on 02/09/2026).
- [27] Falcon Co., Ltd. ごあいさつ [Greetings]. Japanese. Archived from the original on January 16, 2026. Sokoban.jp. 2026. URL: <https://www.sokoban.jp/> (visited on 02/09/2026).

- [28] Shogakukan. “ソフトハウス訪問 [Visiting a Software Company]”. In: *POPCOM (in Japanese)* (June 1984). Interview/Feature regarding Thinking Rabbit, p. 131.
- [29] Atelier Double. 履歴 1989 [*History 1989*]. Japanese. Archived from the original on May 26, 2000. Atelier Double. 2000. URL: <http://www.at-double.co.jp/> (visited on 05/26/2000).
- [30] Carol Ellison. “Special Report: Why Japan Can’t Write Software”. In: *PC Computing* 1.5 (Dec. 1988), pp. 113–114.
- [31] Falcon Co., Ltd. 倉庫番スマート [*Sokoban Smart*]. Japanese. Archived from the original on December 31, 2025. Sokoban.jp. 2026. URL: <https://www.sokoban.jp/> (visited on 02/13/2026).
- [32] Falcon Co., Ltd. ラインナップ [*Lineup*]. Japanese. Archived from the original on February 13, 2026. Sokoban.jp. 2026. URL: <https://www.sokoban.jp/> (visited on 02/13/2026).
- [33] Junji Tanaka, Hiroyuki Himabayashi, and M. Ishii. *THE 倉庫番 [The Sokoban]*. Japanese. Hiroyuki Imabayashi is often credited as Himabayashi in certain 1980s Western indices. SCALE, 1986. ISBN: 4-88239-606-8.
- [34] Shogakukan. “倉庫番トラの巻 [The Sokoban Strategy Guide]”. In: *POPCOM (in Japanese)* (July 1984). Special strategy feature for Sokoban, p. 101.
- [35] Chukyo Television Broadcasting. 中京テレビ、累計 410 万本以上の実績を持つ名作ゲームとコラボ新規ゲーム『チュウキョ〜くんの倉庫番』がデータ放送通信コンテンツ「チュウキョ〜くんランド」に誕生! Japanese. Archived from the original on November 19, 2024. @Press. July 2018. URL: <https://www.atpress.ne.jp/news/160824> (visited on 02/09/2026).
- [36] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. “Deep Learning”. In: *Nature* 521.7553 (2015), pp. 436–444.
- [37] Stephen A Cook. “The complexity of theorem-proving procedures”. In: *Logic, automata, and computational complexity: The works of Stephen A. Cook*. 2023, pp. 143–152.
- [38] Andrew Myers. *XSokoban*. <https://www.cs.cornell.edu/andru/xsokoban.html>. Accessed: 2026-03-08.
- [39] Andreas Junghanns and Jonathan Schaeffer. “Sokoban: A challenging single-agent search problem”. In: *IJCAI Workshop on Using Games as an Experimental Testbed for AI Research*. Morgan Kaufmann Publishers San Francisco, CA. 1997, pp. 27–36.
- [40] Andreas Junghanns and Jonathan Schaeffer. “Sokoban: Evaluating Standard Single-Agent Search Techniques in the Presence of Deadlock”. In: *Advances in Artificial Intelligence. CSCI 1998*. Vol. 1418. Lecture Notes in Computer Science. Springer, Berlin, Heidelberg, 1998, pp. 1–15. DOI: 10.1007/3-540-64575-6_1.
- [41] André Pereira, Marcus Ritt, and Luciana Buriol. “Finding optimal solutions to Sokoban using instance dependent pattern databases”. In: *Proceedings of the International Symposium on Combinatorial Search*. Vol. 4. 1. 2013, pp. 141–148.
- [42] David Ha and Jürgen Schmidhuber. “World Models”. In: (2018). DOI: 10.5281/ZENODO.1207631. URL: <https://zenodo.org/record/1207631>.

- [43] Yaron Shoham and Gal Elidan. “Solving sokoban with forward-backward reinforcement learning”. In: *Proceedings of the International Symposium on Combinatorial Search*. Vol. 12. 1. 2021, pp. 191–193.
- [44] J. Hoffmann and B. Nebel. “The FF Planning System: Fast Plan Generation Through Heuristic Search”. In: *Journal of Artificial Intelligence Research* 14 (May 2001), pp. 253–302. ISSN: 1076-9757. DOI: 10.1613/jair.855. URL: <http://dx.doi.org/10.1613/jair.855>.
- [45] Ilya Loshchilov and Frank Hutter. “Decoupled weight decay regularization”. In: *arXiv preprint arXiv:1711.05101* (2017).
- [46] Peter J Huber. “Robust estimation of a location parameter”. In: *Breakthroughs in statistics: Methodology and distribution*. Springer, 1992, pp. 492–518.